

VIVADO使用教程

目录

- ① 1. Vivado软件概述
- ② 2. Vivado软件使用基本流程
- ③ 3. Vivado约束设计基本流程
- ④ 4. Vivado开发的注意事项
- ⑤ 附录1：SDC介绍
- ⑥ 附录2：ISE时序约束设计

目录

- ◎ 1. *Vivado* 软件概述
- ◎ 2. Vivado 软件使用基本流程
- ◎ 3. Vivado 约束设计基本流程
- ◎ 4. Vivado 开发的注意事项
- ◎ 附录1：SDC 介绍
- ◎ 附录2：ISE 时序约束设计

Vivado软件概述

◎ Vivado的发展：

- Vivado从2008年5月开始研发；
- 2011年4月开始进行超过100个试用用户的β测试；
- 目前版本号为2014.2。

◎ Vivado将关注点集中在C2H、高层次综合等技术方面，仿真内核的性能得到优化，宣称其仿真速度是ISE的3倍。如果使用硬件联合仿真，可以将速度提高100倍。官方宣称Vivado比竞争对手速度快4倍，而消耗的资源是对手的50%。

Vivado软件概述

- ◎ Vivado与ISE相比，发生的改进：
 - 1.数据格式更统一；
 - 2.constraint文件采用了SDC格式，兼容业界标准；
 - 3.Vivado的ECO和TCL脚本功能更强大；
 - 4.推出了针对C开发的High Level Synthesis和针对DSP的System Generator；
- ◎ 支持的器件系列有：
 - Virtex-7\Kintex-7\Artix-7\Zynq 7000
 - Virtex UltraScale\Kintex UltraScale

Vivado软件概述——Tcl示例（1）

- 对一个flip-flop进行位置更改的ECO：
 - *startgroup*
 - *place_cell {q_reg[2]} SLICE_X3Y2/C5FF*
 - *endgroup*
 - *read_checkpoint -incremental old_post_route.dcp*
 - *report_incremental_resue*
 - *route_design*

Vivado软件概述——Tcl示例（2）

● 一个没有建立工程的vivado脚本：

- *#Assemble the Design Source files*
- *read_verilog [glob ../src/*.v]*
- *read_vhdl [glob ../src/*.vhd]*
- *read_edif ../netlist/black_blox.edf*
- *read_xdc ../constraint/top.xdc*
- *#Run Synthesis and Implementation*
- *synth_design -top top -part xc7k70tfbg676-2*
- *write_checkpoint -force post_synth.dcp*
- *opt_design*
- *place_design*
- *write_checkpoint -force post_place.dcp*
- *route_design*
- *write_checkpoint -force post_route.dcp*
- *#Generate Reports*
- *report_timing_summary -file timing_summary.rpt*
- *.....*
- *#Generate Bit File*
- *write_bitstream -file top.bit*

Vivado软件概述

以SPI模块为例，比较一下两个软件的综合布线后的资源使用情况。

- 该SPI模块用于1431、1451项目中，同时输出两路SPI信号。工程选择的比较芯片为xc7vx485tffg1157-1，时钟设为50MHz。
- Vivado2014.2的利用率报告：

Resource	Utilization	Available	Utilization %
Slice LUTs	326	303600	0.11
Slice Registers	394	607200	0.06
I/O	78	702	11.11
Clocking	1	32	3.12

- ISE14.7的利用率报告：
- 两者比较：

	LUT	DFF
Vivado	326	394
ISE	395	271

Device Utilization Summary			
Slice Logic Utilization		Used	Available
Number of Slice Registers		271	607,200
Number used as Flip Flops		271	
Number used as Latches		0	
Number used as Latch-thrus		0	
Number used as AND/OR logics		0	
Number of Slice LUTs		395	303,600
Number used as logic		394	303,600
Number using 06 output only		298	
Number using 05 output only		3	
Number using 05 and 06		93	
Number used as ROM		0	
Number used as Memory		0	130,800
Number used exclusively as route-thrus		1	
Number with same-slice register load		0	
Number with same-slice carry load		1	
Number with other load		0	

Vivado软件概述

- 以FFT为例，比较两个软件的综合布线后的资源使用情况。

- 该FFT核配置为：1024点，13位输入，18位输出，蝶型运算因子18位，选择的比较芯片为XC7K325T-2FFG900，时钟设为250MHz。
- 两种工具软件最后的资源利用率比较：

	DFF	LUT	DSP	RAM
Vivado2014.2	29,149	17,201	192	276
ISE14.7	25,746	21,673	192	252

- Vivado会使用较少的LUT，而多用一些DFF。在SLICE中一个LUT对应两个DFF，从整体上看优化了LUT的使用。

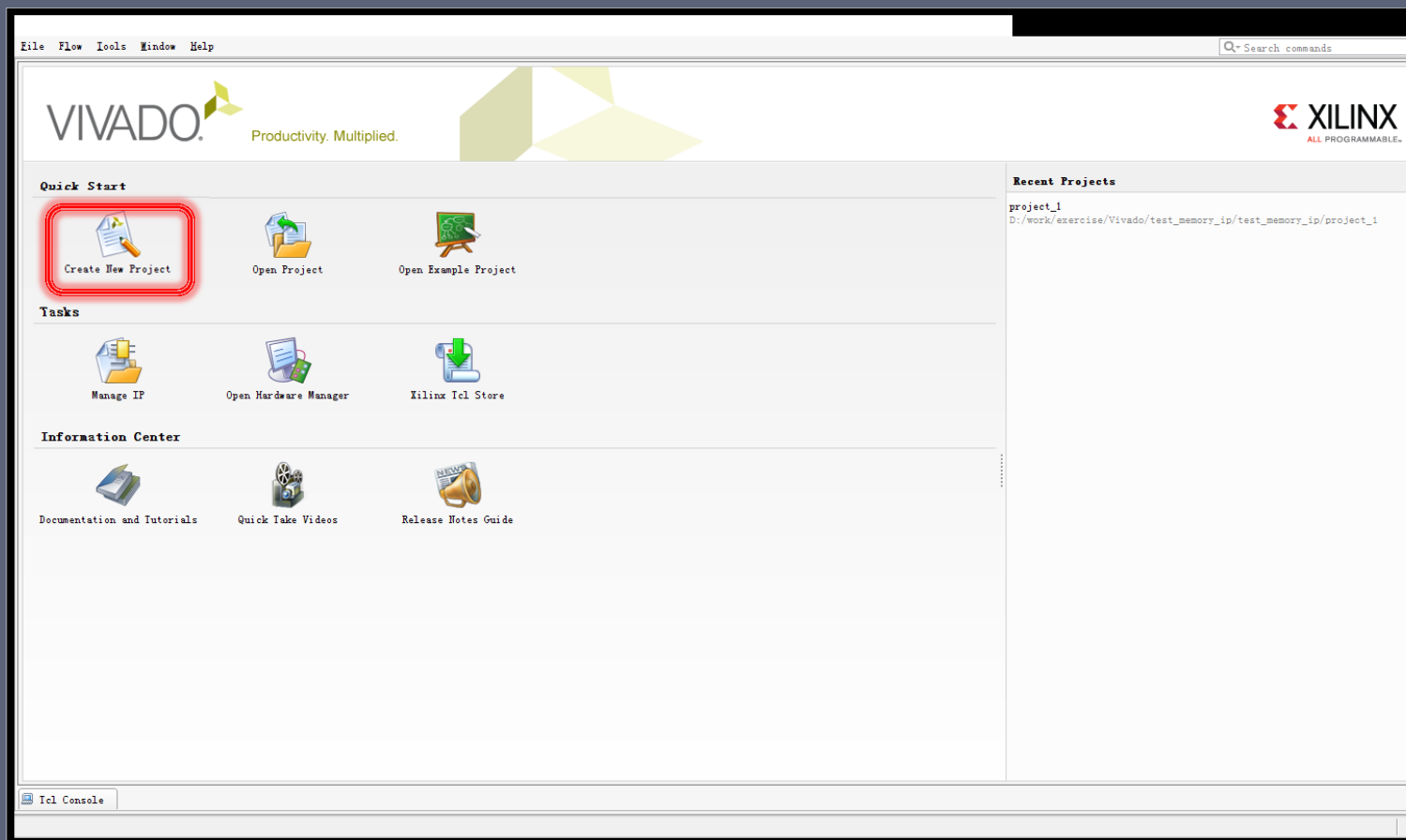
目录

- ◎ 1. Vivado软件概述
- ◎ 2. *Vivado*软件使用基本流程
- ◎ 3. Vivado约束设计基本流程
- ◎ 4. Vivado开发的注意事项
- ◎ 附录1：SDC介绍
- ◎ 附录2：ISE时序约束设计

Vivado软件使用基本流程

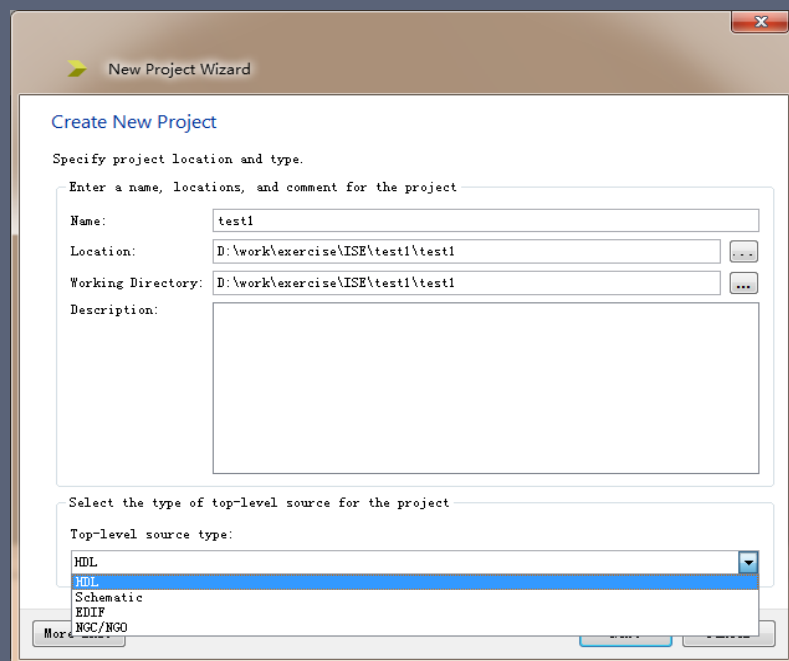
- ◎ 新建工程的基本流程；
- ◎ 导入ISE工程的基本流程及注意事项；
- ◎ IP生成；
- ◎ 功能仿真；
- ◎ 综合操作；
- ◎ 实现操作；
- ◎ 生成配置文件。

新建工程的基本流程

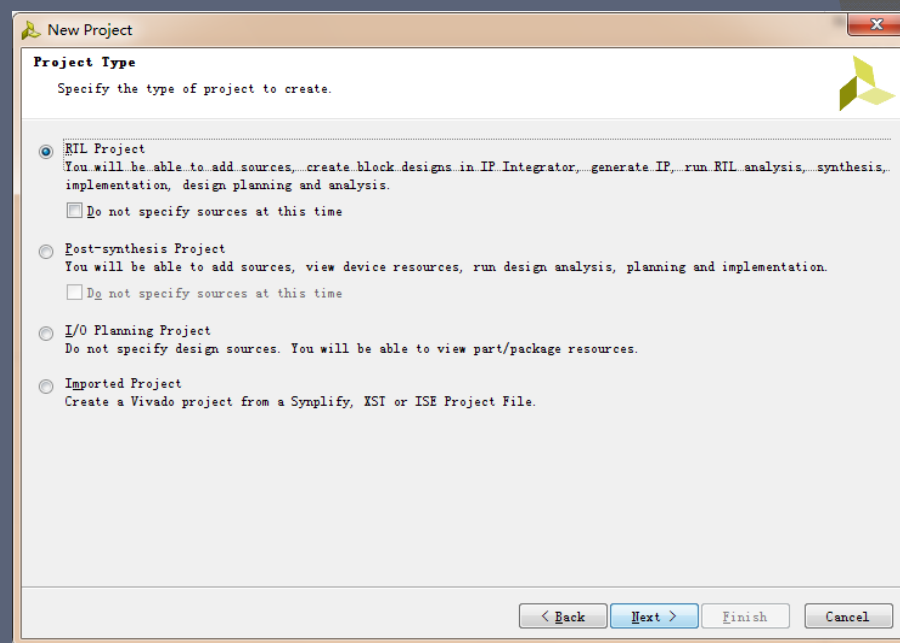


新建工程的基本流程

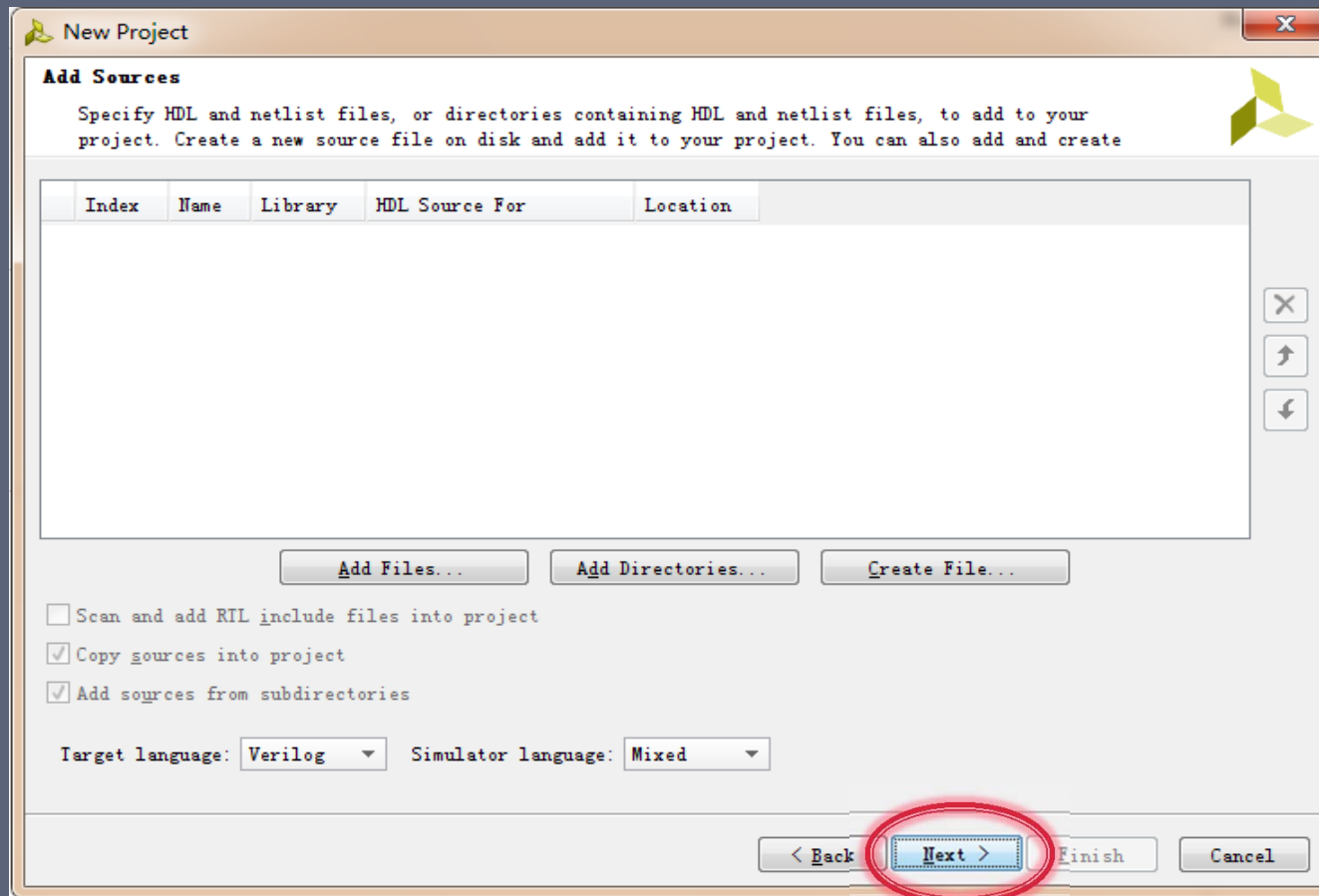
ISE14.7新建工程界面



Vivado2014.2新建工程界面

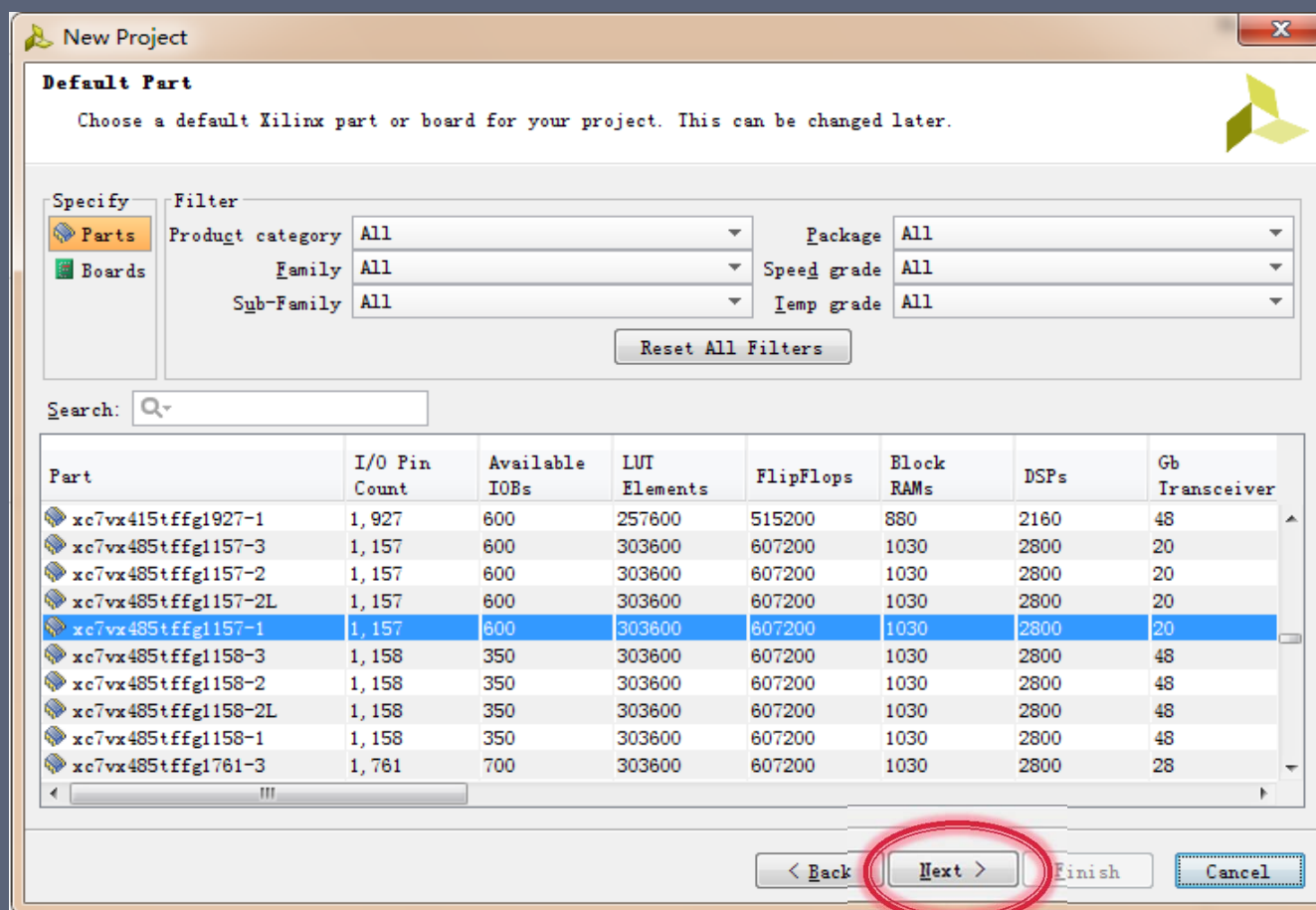


新建工程的基本流程——添加资源

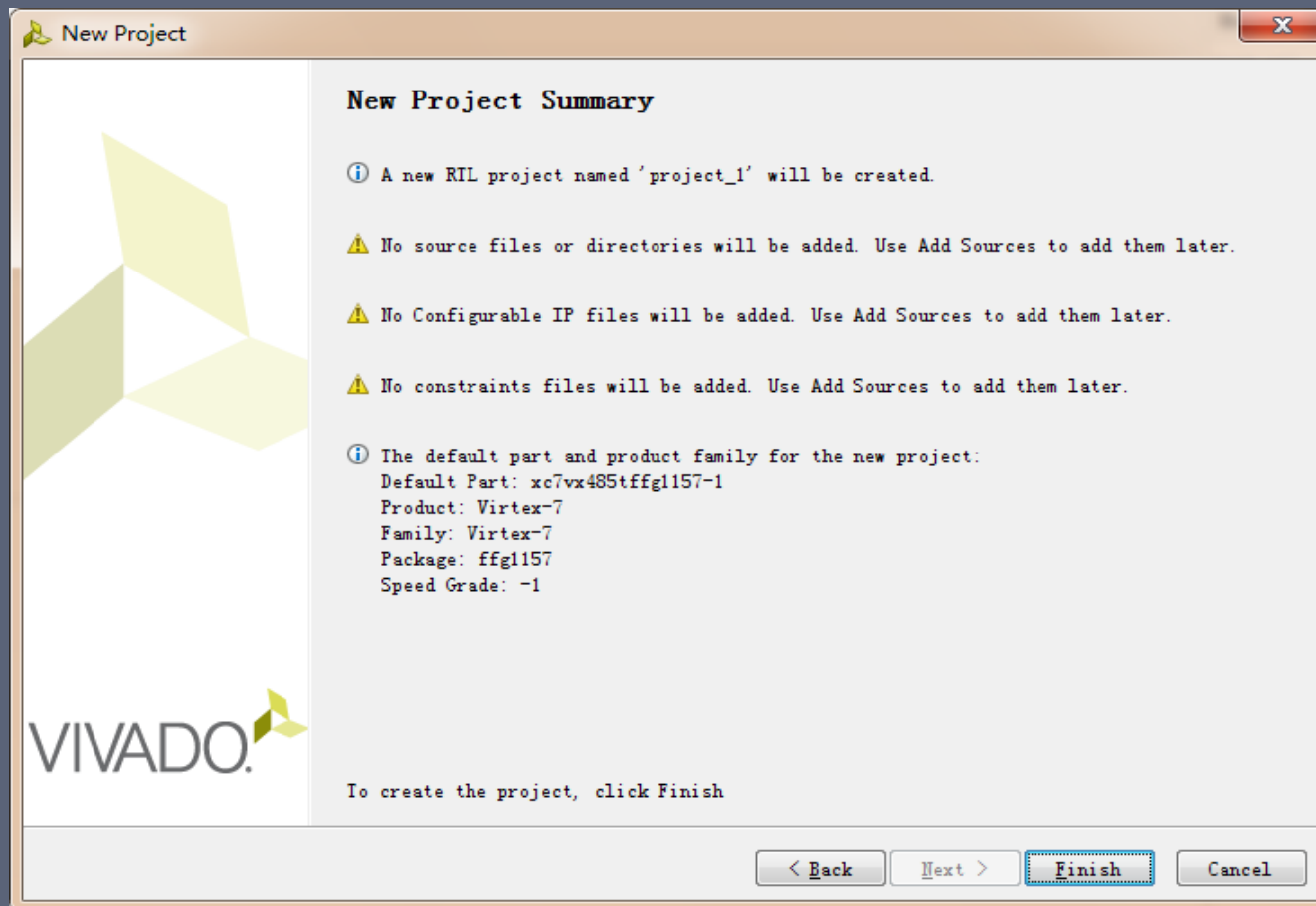


添加代码、
IP、网表
等的界面
基本相同。

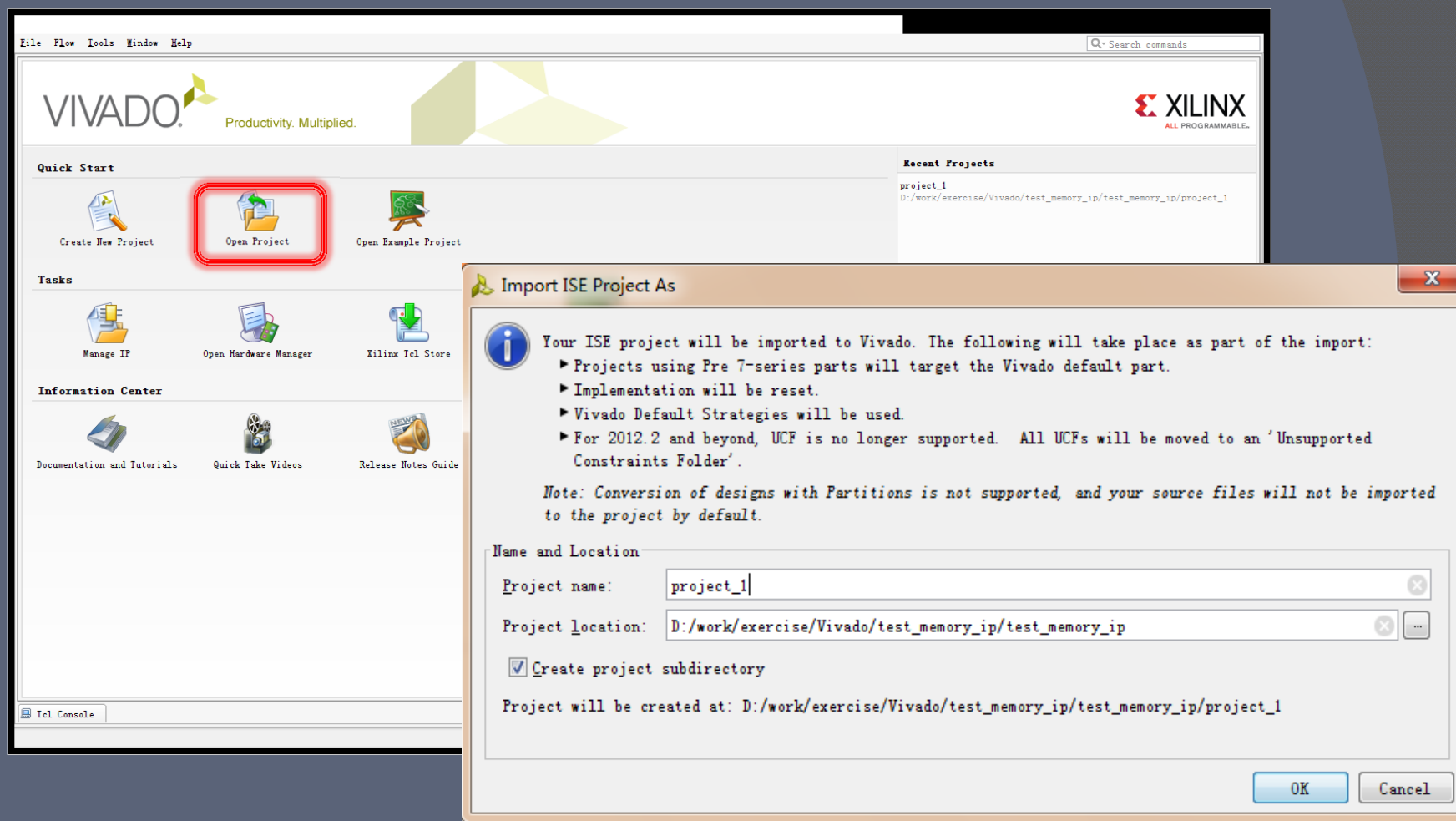
新建工程的基本流程——选择器件



新建工程的基本流程——工程总概



导入ISE工程的基本流程



导入ISE工程的基本流程

◎ 迁移工程所将发生的变化有：

- 1) 使用7系列之前的器件将会被替换成Vivado所默认的对应该器件；
- 2) 实现步骤将会被重置；
- 3) 将采用Vivado默认的策略（包括一些用法、算法、格式规定、优化方法等）
- 4) 从2012.2版本开始，Vivado将不再支持UCF文件。所有的UCF将被移入“不支持的约束文件夹”

◎ 注意：不支持对具有Partition的设计进行转化，默认情况下源文件将无法导入到工程中。

导入ISE工程的基本流程

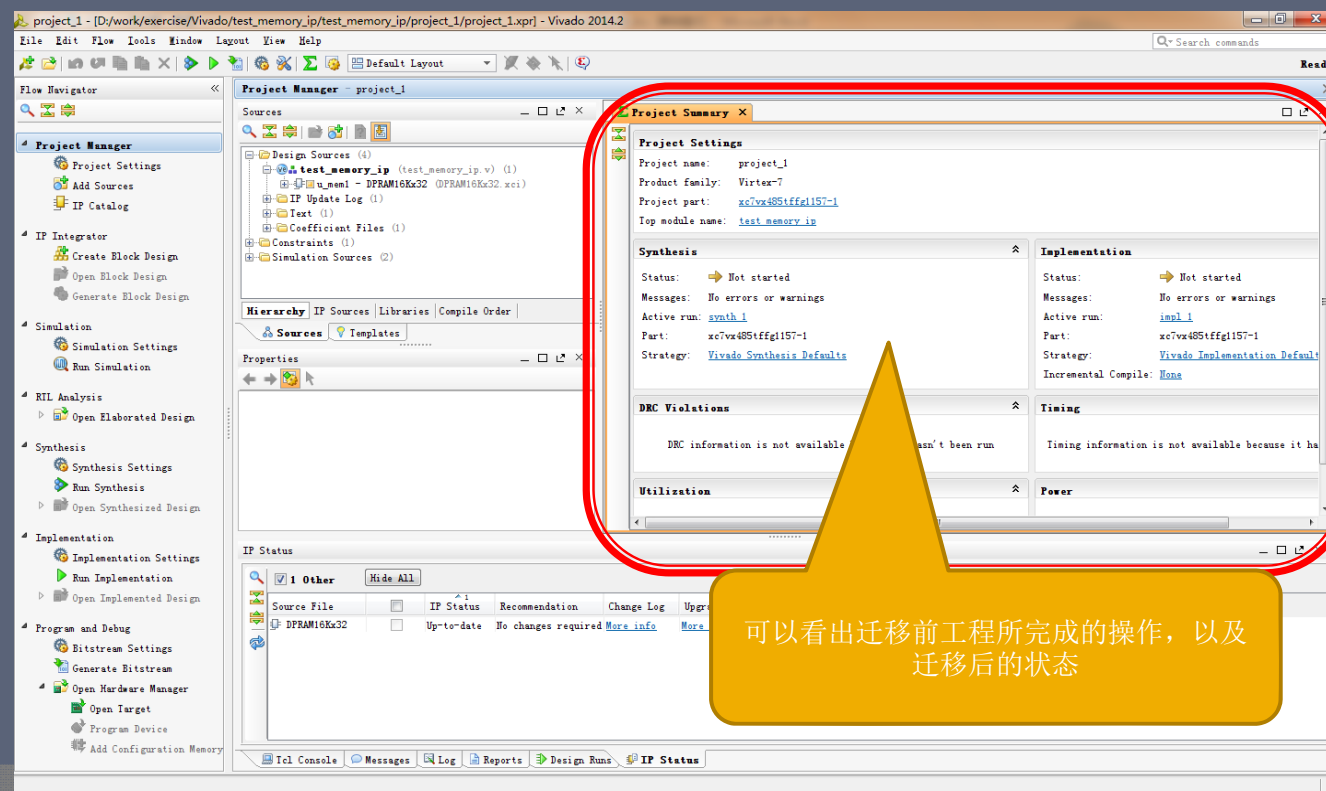
- ◎ 迁移之后将会产生一个报告文档，里面将会列出迁移过程所产生的各种变化，以及各种注意事项。
- ◎ 报告文档分为三个部分：
 - 1、ISE工程信息汇总
 - 2、未导入信息汇总
 - 3、导入信息汇总

导入ISE工程的基本流程

- 文档中提到：对于约束文件的导入，UCF文件会被加入到Vivado的工程中，但是被识别为需要手动将其设置为“目标约束文件”，具体方法是：在“source”窗口中，右键单击约束文件，选择“Set Target UCF”，或者使用Tcl语句"`set_property target_constrs_file <filename> <fileset>`"。
- 实际上在2014.2版本中已经不可这样导入了，UCF文件格式已经被彻底放弃了。

导入ISE工程的基本流程

- 迁移报告对工程操作的几项流程环节都分别进行了汇总，介绍了每个环节所执行的操作和采取的策略。这里将不再采用ISE的各种策略。Vivado的各种策略，可以在“Tools → Options → Strategies”中看到具体内容。



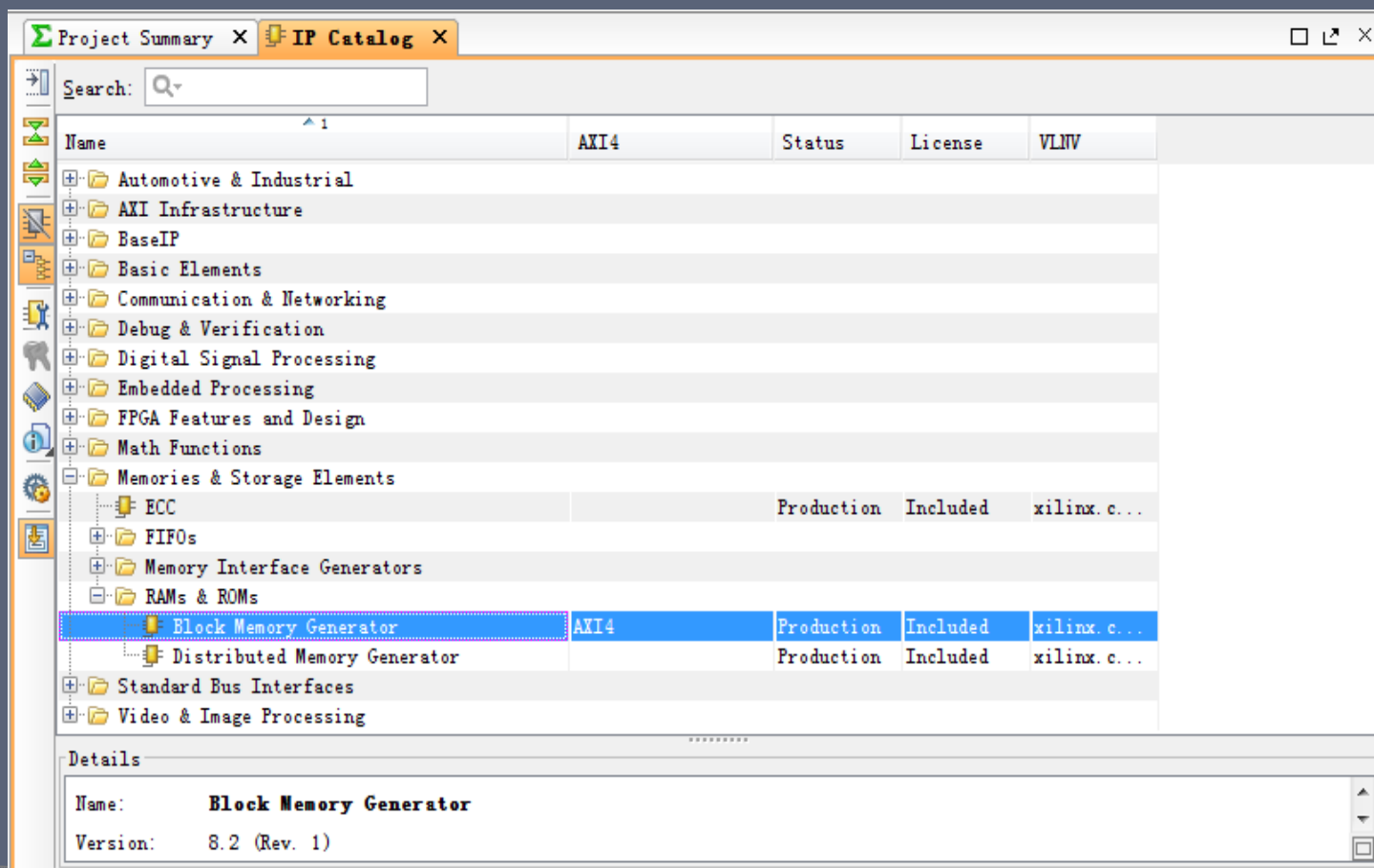
导入ISE工程的基本流程

- Vivado的迁移仍然存在不足，基本上全部的IP需要重新生成，就连DCM的IO名称在两款软件中都有差异，因此迁移工作基本上只对应HDL代码自身而言是平滑的。



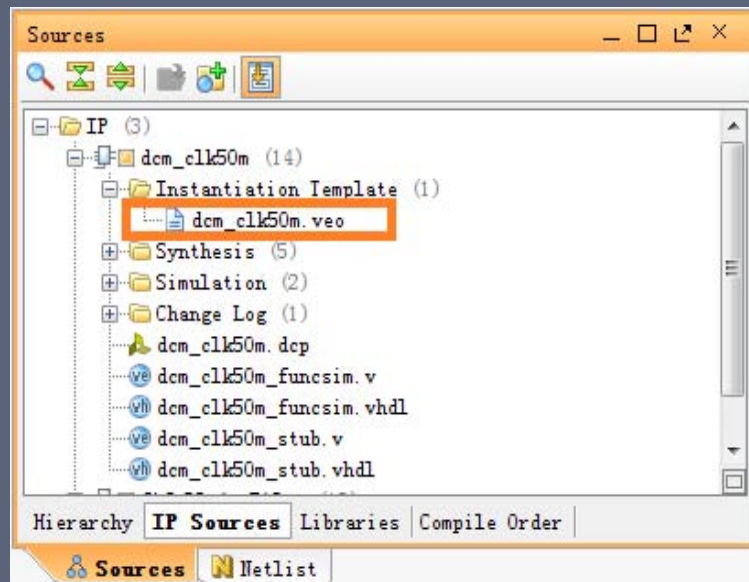
IP生成

● IP Catalog窗口



IP生成

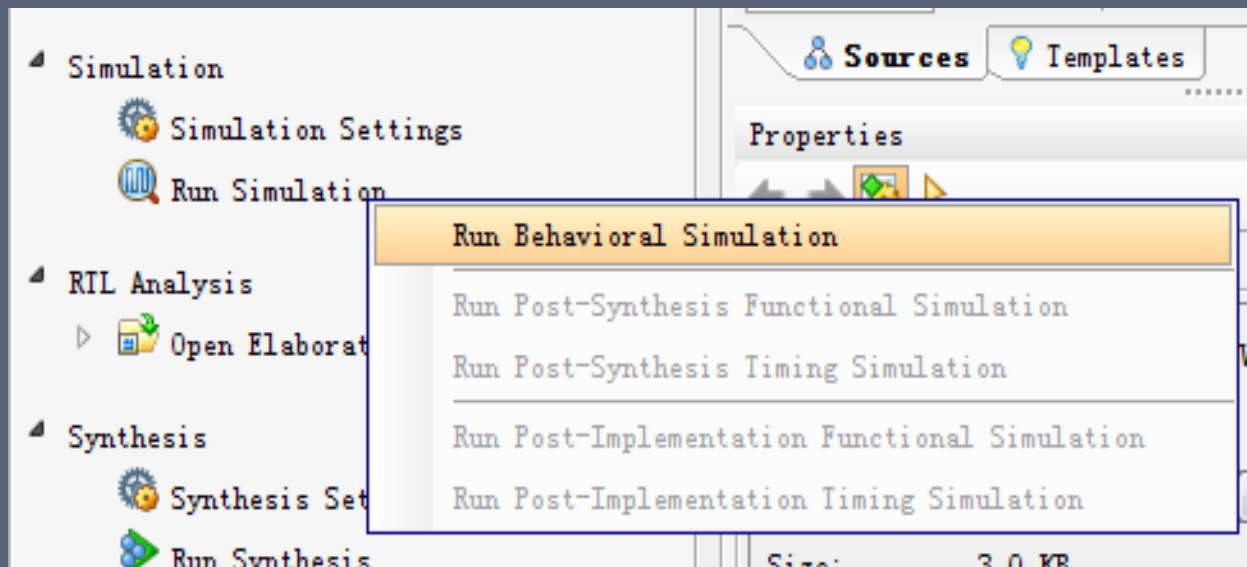
- 生成的IP可以在source→IP source窗口中对应IP的Instantiation Template目录中找到.veo文件，在文件中能够找到instance例化时的模板，节省编辑时间。



```
//----- Begin Cut here for INSTANTIATION Template -----// INST_TAG
SPRAM4KX32 your_instance_name (
    .clka(clka),      // input wire clka
    .ena(ena),        // input wire ena
    .wea(wea),        // input wire [3 : 0] wea
    .addra(addra),    // input wire [11 : 0] addra
    .dina(dina),      // input wire [31 : 0] dina
    .douta(douta)     // output wire [31 : 0] douta
);
// INST_TAG_END ----- End INSTANTIATION Template -----
```

功能仿真

- Vivado提供了类似Modelsim的仿真工具。



功能仿真

The screenshot displays the Vivado 2014.2 Behavioral Simulation interface for a project named `test_spi`. The main window shows the simulation results for the `test_spi_top` design unit. The interface is divided into several panes:

- Scopes:** Lists the design units and their block types. The `test_spi_top` unit is highlighted, and a red box with the text "仿真目标的层次结构示意" (Diagram of the simulation target hierarchy) points to it.
- Objects:** Lists the objects in the simulation. The `pll_ref_wrd[31:0]` object is highlighted, and a red box with the text "在这里选择所要观察的目标" (Select the target to be observed here) points to it.
- Source File Properties:** Shows the properties of the selected source file, including the design unit, block type, and file path.
- Tcl Console:** Displays the simulation results, including the time resolution (1 ps), simulation completion status, and launch time.
- Waveform:** Shows the timing diagram for the selected objects. The `pll_ref_wrd[31:0]` signal is highlighted, and a red box with the text "在这里选择所要观察的目标" (Select the target to be observed here) points to it.

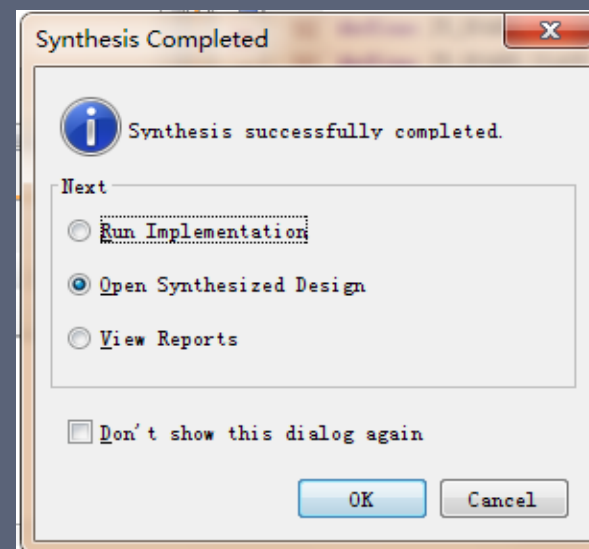
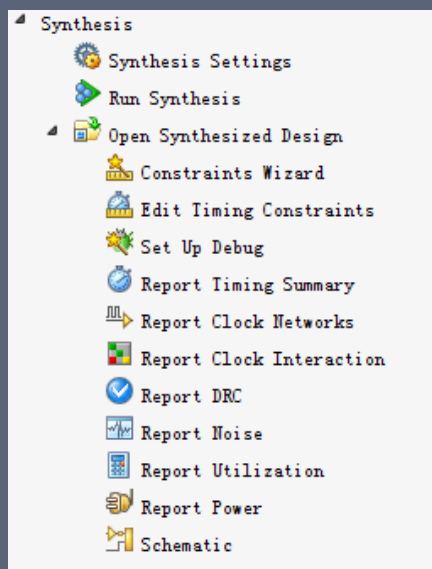
The waveform shows the signal values over time, with a vertical yellow line indicating the current simulation time at 76,209.948 ns. The signal values are displayed in hexadecimal and decimal formats.

功能仿真

- 将需要的观测对象添加到波形窗口后，需要点击  对波形复位，然后点击  或者   重新产生波形。
- 如果修改了代码，则需要点击 ，重新启动仿真操作。
- 当完成了综合和实现的环节可以对这些环节中产生的网表进行功能和时序上的仿真。这里时序仿真只支持Verilog格式的网表，如果需要可以使用“write_verilog”产生网表。
- 仿真的时间分辨率官方建议是1ps。有些赛灵思原语组件（例如DCM）在功能或时序仿真中都需要1ps的分辨率才能正常工作。

综合操作

- 在Flow Navigator中选择Run Synthesis。如果没有报错的话，会出现下面右侧的对话框。

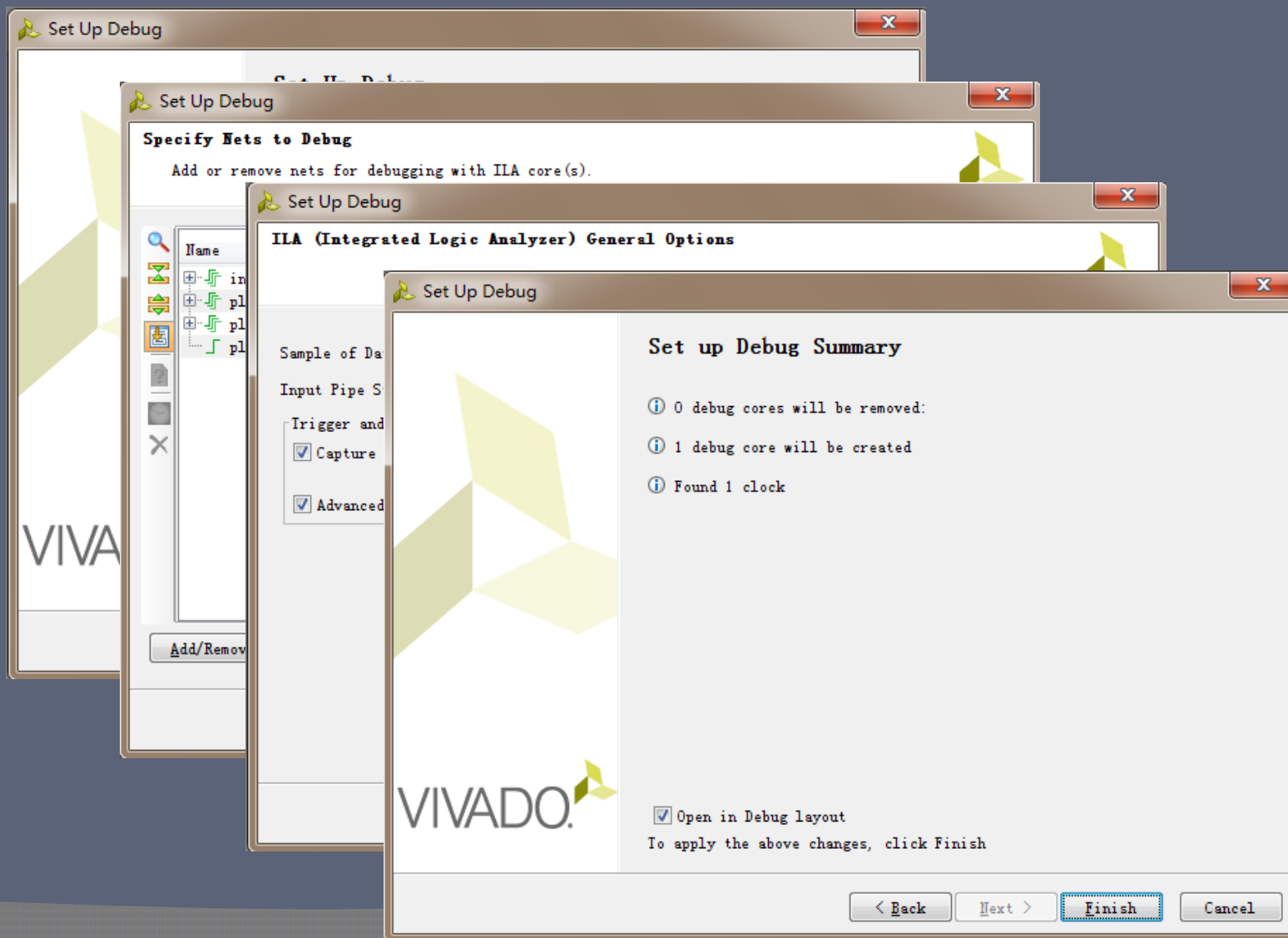


- 需要手动选择是否打开综合后的输出结果，或者可以直接进行实现环节的操作。

综合操作

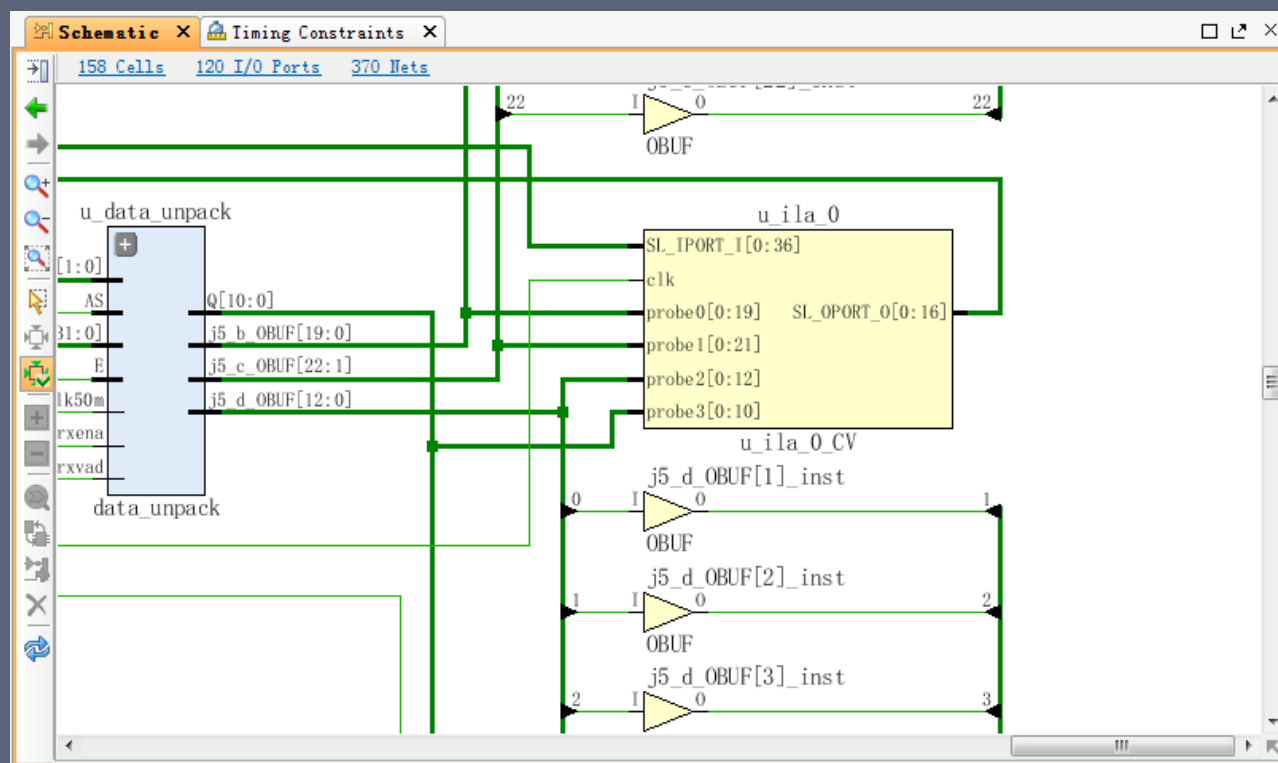
- 综合之后可以进行debug的设置，也就是相当于ISE中CDC文件的设置。
- 对于CDC文件，Vivado已经不再支持。关于系统中debug的设置信息，将保存在.xdc文件中。

综合操作



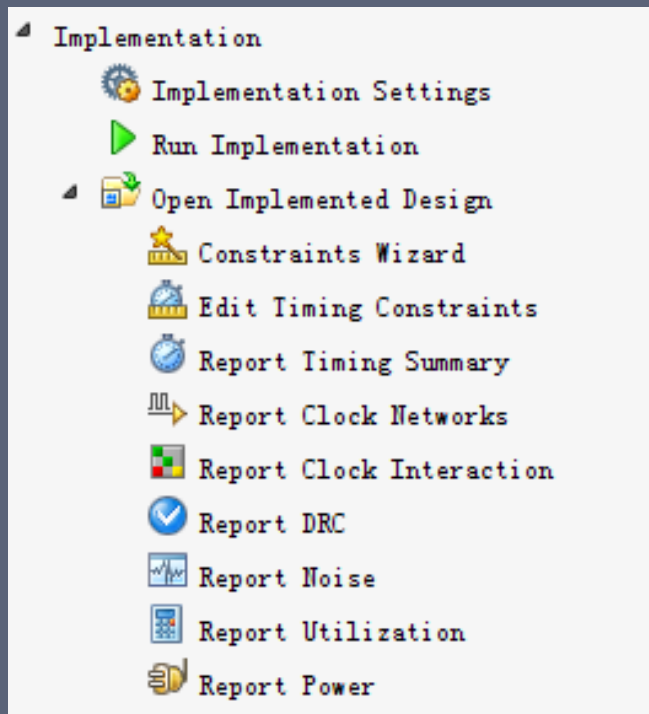
综合操作

- 设置完成后可以在原理图中看到加入的ILA等。



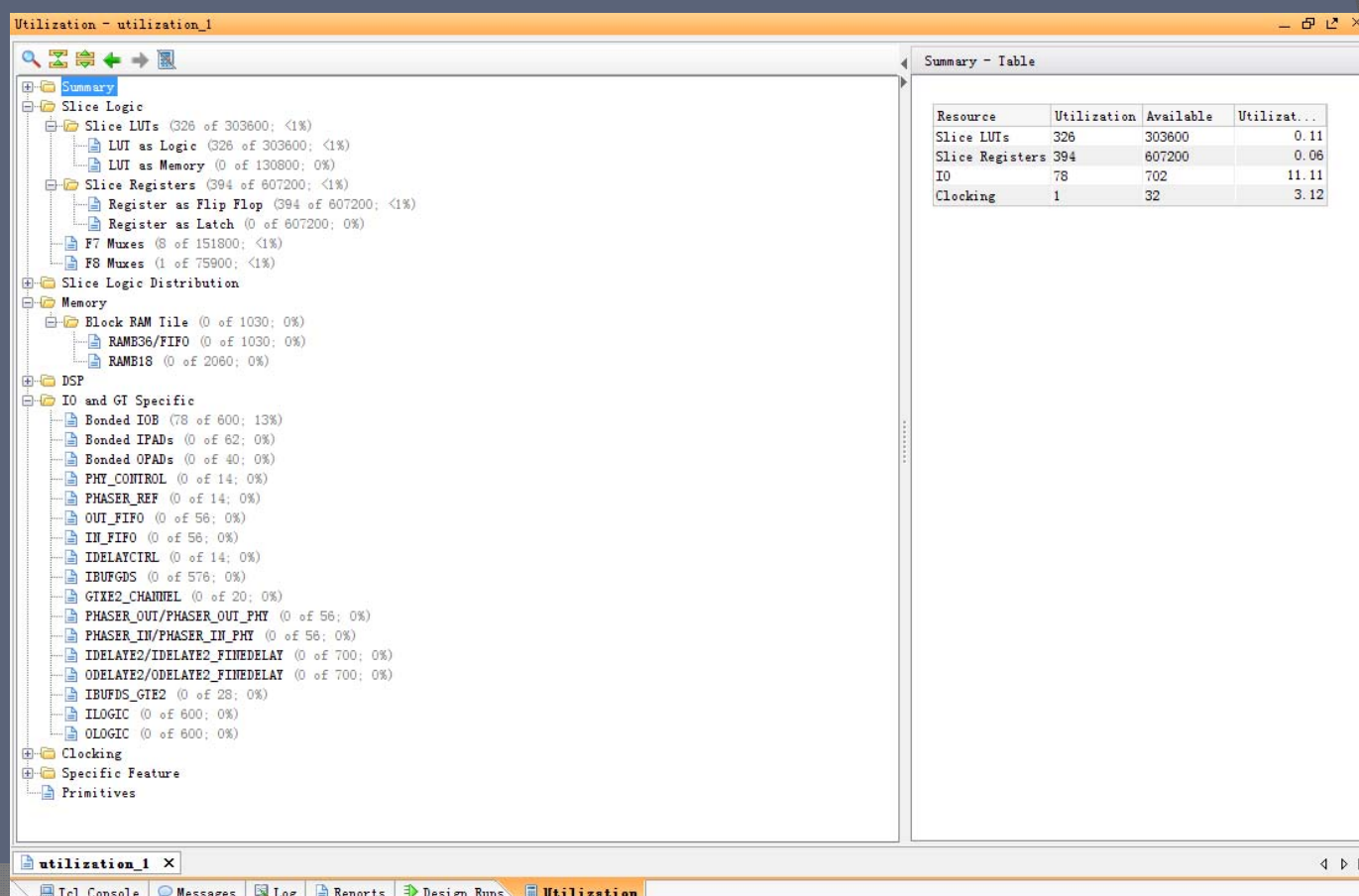
实现操作

- 综合完成之后需要添加约束条件，检查一切正常之后就可以开始实现了。



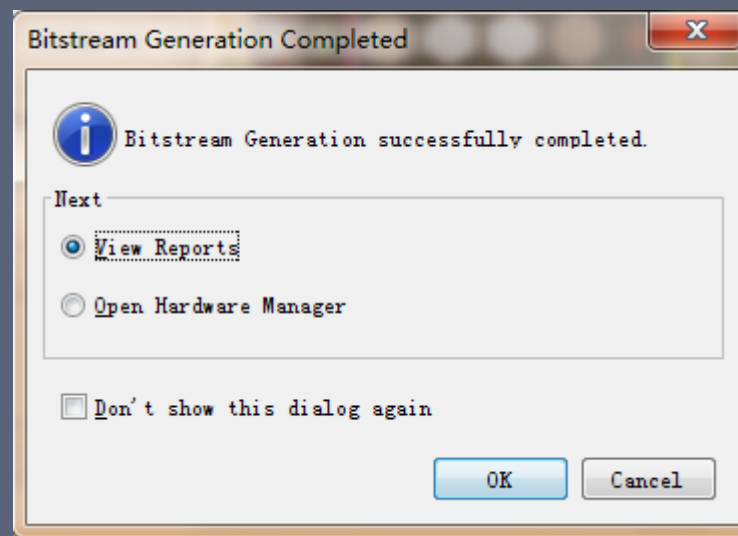
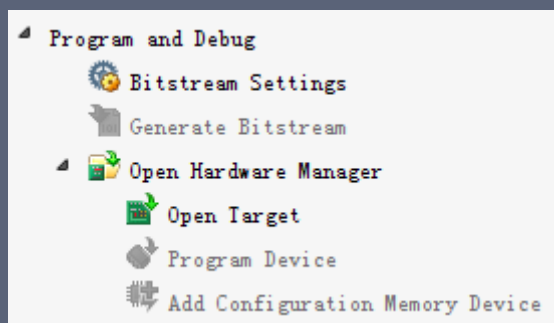
实现操作

- 实现完成后需要做各种分析报告，具体和ISE基本类似。最后检查资源利用率的情况。



生成配置文件

- 确认实现环节的结果无误之后，就可以生成位流文件了。

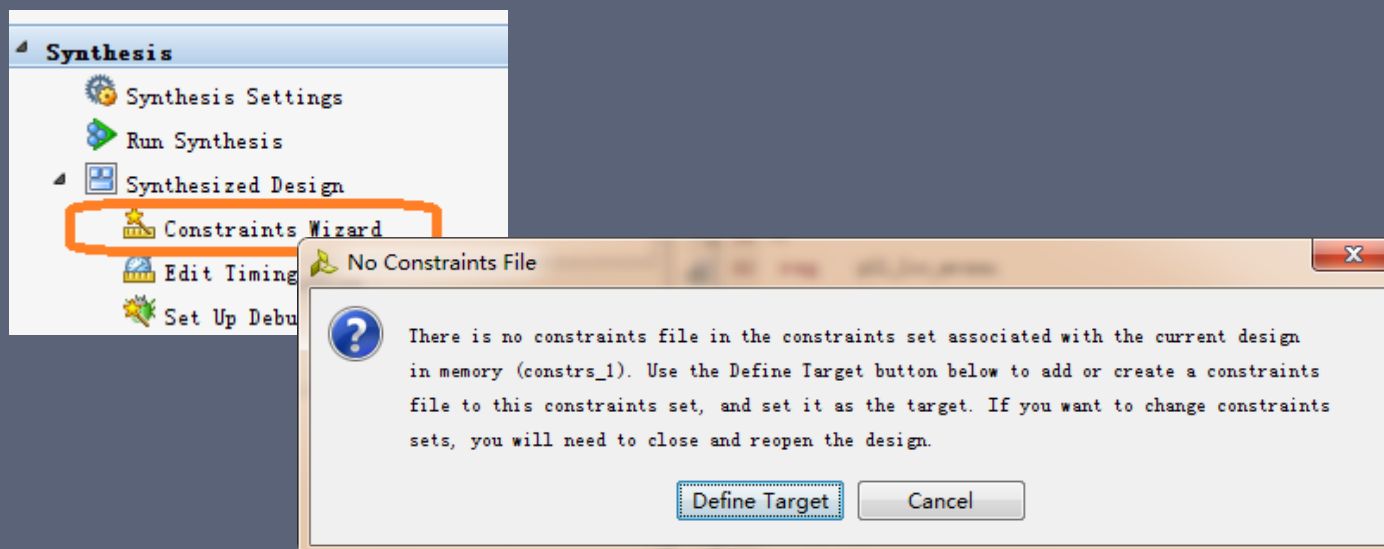


目录

- ◎ 1. Vivado软件概述
- ◎ 2. Vivado软件使用基本流程
- ◎ 3. *Vivado约束设计基本流程*
- ◎ 4. Vivado开发的注意事项
- ◎ 附录1：SDC介绍
- ◎ 附录2：ISE时序约束设计

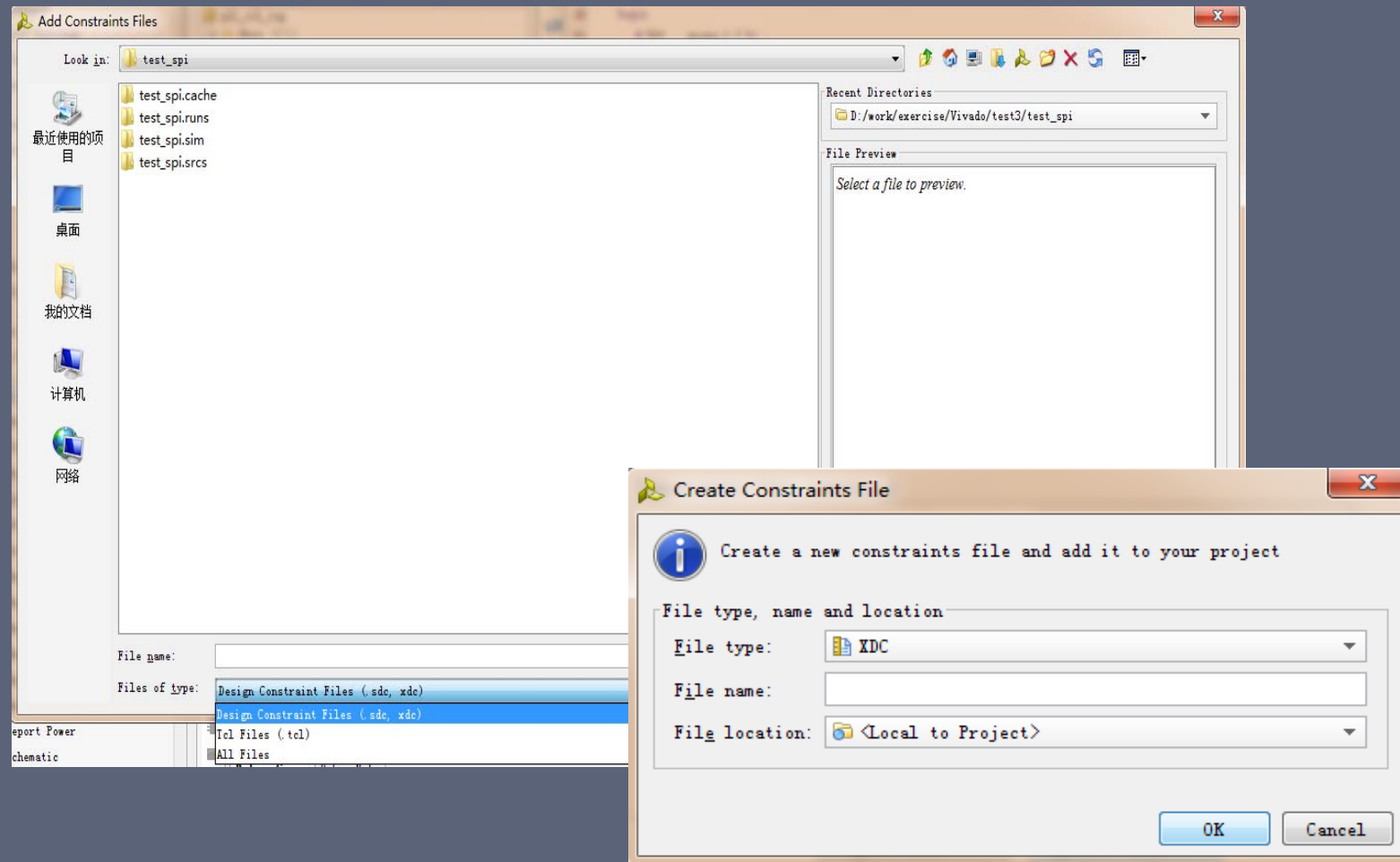
Vivado约束设计基本流程

- 当完成综合之后，就可以添加约束了。



- 在Vivado2014.2版本中，原来的UCF文件是无法继续使用的。新的约束文件中所采用的语法格式和ISE中的格式差别较大，而和SDC格式基本相同。

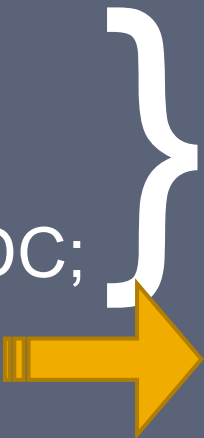
Vivado约束设计基本流程



Vivado约束设计基本流程

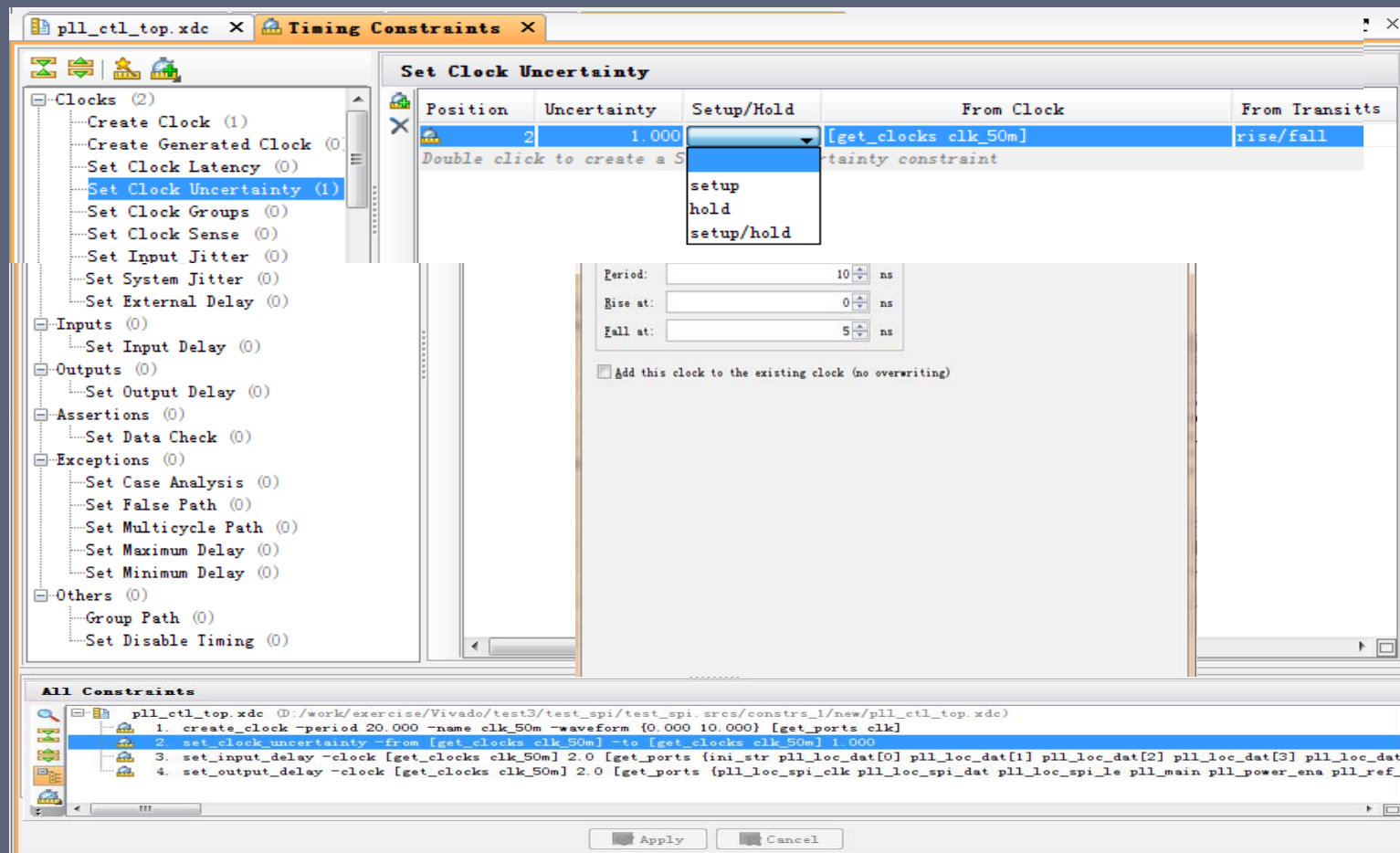
● XDC文件中的时序约束主要包括如下四个部分：

- Create Clocks;
- Input /Output Delays;
- Clock Groups and CDC;
- Timing Exceptions



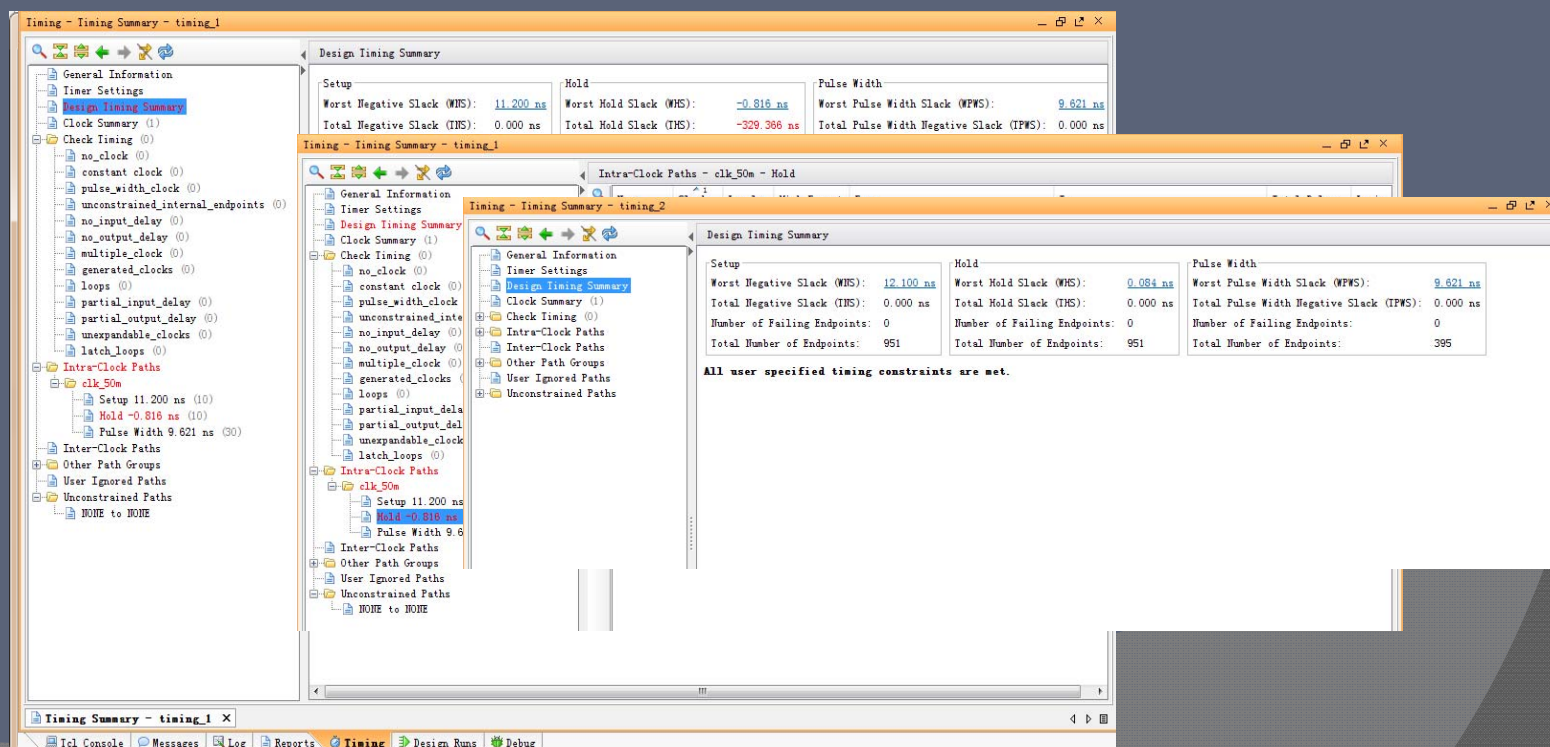
主时钟; create_clock
虚拟时钟; create_clock
衍生时钟;
create_generated_clock
时钟路径确定性; set_false_path
最大、最小延迟; set_max_delay
set_clock_uncertainty
输入、输出延迟; set_min_delay
多周期路径; set_multicycle_path
案例分析; set_case_analysis
set_output_delay
时钟组; set_clock_groups
set_disable_timing

Vivado约束设计基本流程



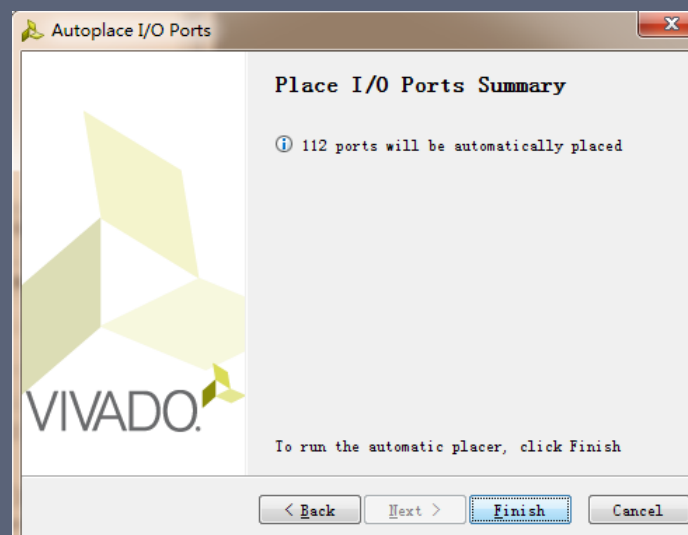
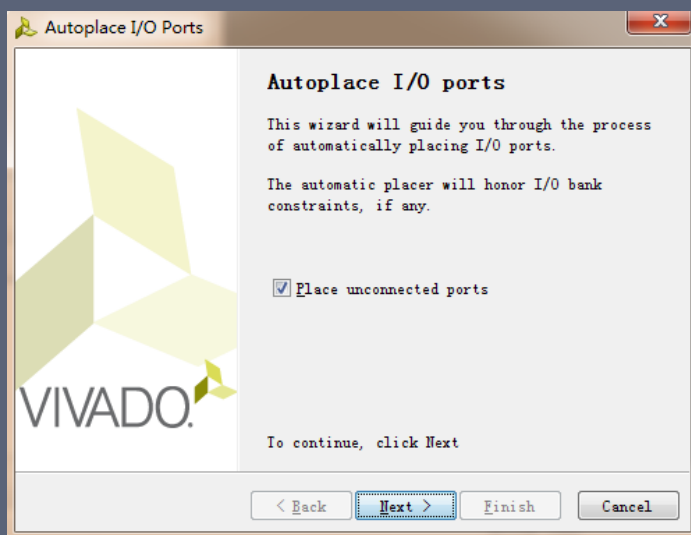
Vivado约束设计基本流程

- 设置完毕后一定要记得进行保存，否则这些参数只保存在内存中，一旦关闭工程就全部丢失了。
- 之后需要重新进行综合，进行时序方面的检查。运行 Reporting Timing Summary, 可以看到目前时序上存在的问题。



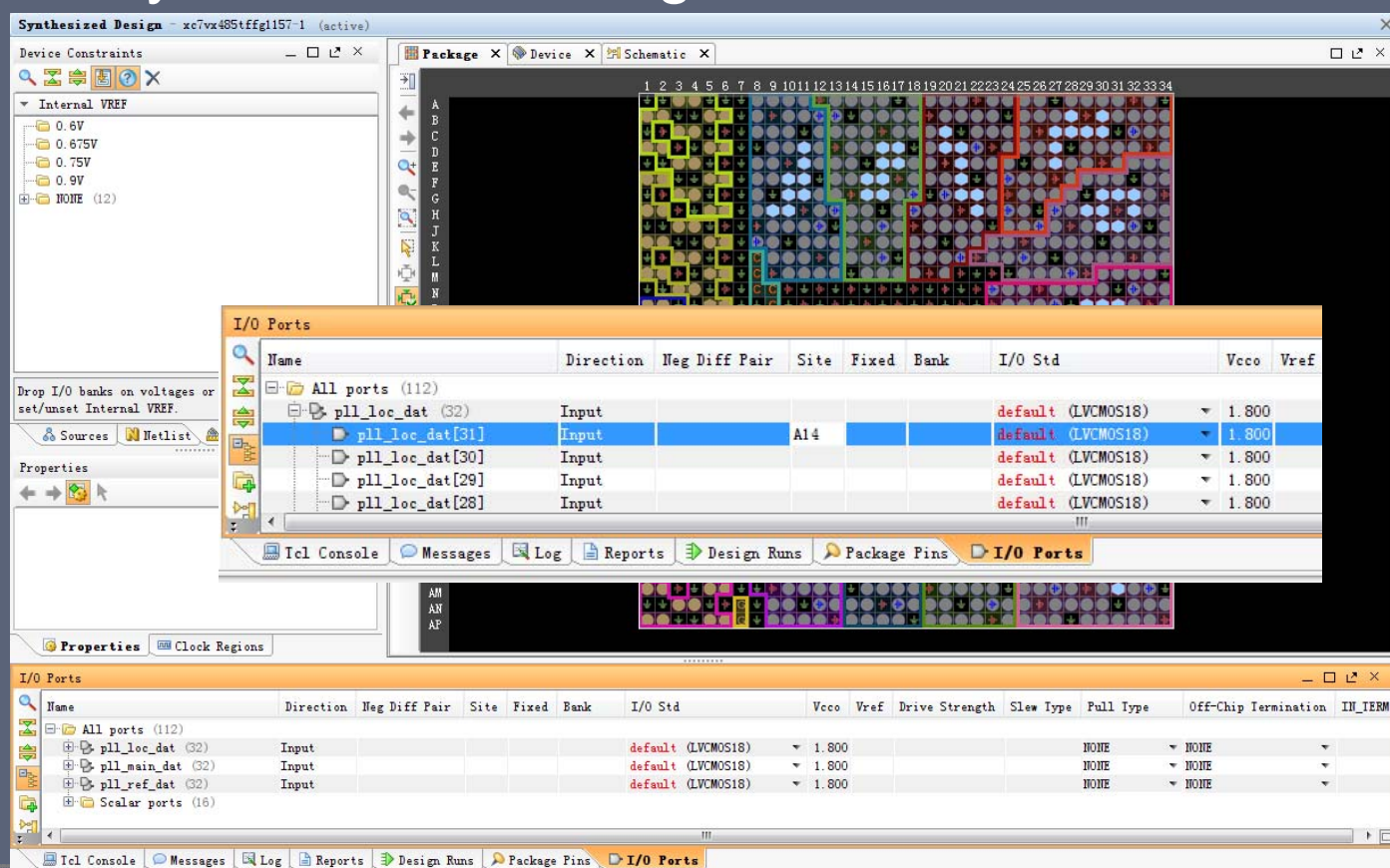
Vivado约束设计基本流程

- Vivado将约束分为时序约束和物理约束。
 - 物理约束主要是指管脚的使用。包括管脚的映射关系和电平标准，以及诸如上拉、下拉等的属性参数。
- 可以使用自动分配管脚工具，在“Tools→I/O Planning”里面可以找到。

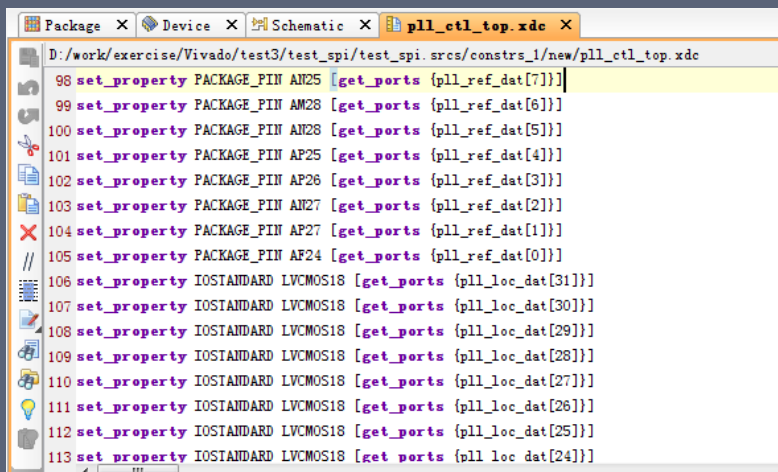


Vivado约束设计基本流程

- 对照板卡的具体情况，手动分配管脚。选择“Layout-> I/O Planning”。



Vivado约束设计基本流程



```
D:/work/exercise/Vivado/test3/test_spi/srcs/constrs_1/new/pll_ctl_top.xdc
98 set_property PACKAGE_PIN AN25 [get_ports {pll_ref_dat[7]}]
99 set_property PACKAGE_PIN AN28 [get_ports {pll_ref_dat[6]}]
100 set_property PACKAGE_PIN AN28 [get_ports {pll_ref_dat[5]}]
101 set_property PACKAGE_PIN AP25 [get_ports {pll_ref_dat[4]}]
102 set_property PACKAGE_PIN AP26 [get_ports {pll_ref_dat[3]}]
103 set_property PACKAGE_PIN AN27 [get_ports {pll_ref_dat[2]}]
104 set_property PACKAGE_PIN AP27 [get_ports {pll_ref_dat[1]}]
105 set_property PACKAGE_PIN AF24 [get_ports {pll_ref_dat[0]}]
106 set_property IOSTANDARD LVCMOS18 [get_ports {pll_loc_dat[31]}]
107 set_property IOSTANDARD LVCMOS18 [get_ports {pll_loc_dat[30]}]
108 set_property IOSTANDARD LVCMOS18 [get_ports {pll_loc_dat[29]}]
109 set_property IOSTANDARD LVCMOS18 [get_ports {pll_loc_dat[28]}]
110 set_property IOSTANDARD LVCMOS18 [get_ports {pll_loc_dat[27]}]
111 set_property IOSTANDARD LVCMOS18 [get_ports {pll_loc_dat[26]}]
112 set_property IOSTANDARD LVCMOS18 [get_ports {pll_loc_dat[25]}]
113 set_property IOSTANDARD LVCMOS18 [get_ports {pll_loc_dat[24]}]
```

☉ ISE:

- *NET "reset_n" LOC = "N2" | IOSTANDARD = LVCMOS33;*

☉ Vivado:

- *set_property PACKAGE_PIN AF24 [get_ports {pll_loc_dat[31]}]*
- *set_property IOSTANDARD LVCMOS18 [get_ports {pll_loc_dat[31]}]。*

目录

- ◎ 1. Vivado软件概述
- ◎ 2. Vivado软件使用基本流程
- ◎ 3. Vivado约束设计基本流程
- ◎ 4. *Vivado开发的注意事项*
- ◎ 附录1：SDC介绍
- ◎ 附录2：ISE时序约束设计

Vivado开发的注意事项

- ◎ 脚本使用方面；
- ◎ 代码风格方面；
- ◎ 资源使用方面；
- ◎ 时序约束方面；
- ◎ 工作流程方面；
- ◎ 器件配置方面。

Vivado使用技巧——脚本使用

- ◎ 工程运行脚本的提取可以使用 “*write_project_tcl*” 命令。
- ◎ 支持一系列针对IP的生成报告和特定格式输出的tcl命令，如 “*write_verilog*” 等。
- ◎ 使用 “*write_sdc*” 时可以加载压缩参数，将产生的约束文件自动进行压缩保存，同时 “*read_sdc*” 命令可以直接解压缩读取信息。例如：
 - *write_sdc -compress gzip design.sdc.gz*
 - *read_sdc design.sdc.gz*

Vivado使用技巧——代码风格

- ◎ One-hot状态机往往更适于FPGA，因其不需解码。而二进制状态机更加安全。
- ◎ Vivado的官方技术文档中多次强调建议使用同步复位，使用高电平做类似中断等控制信号的有效电平。
 - *always @ (posedge clock or posedge rst)*
 - 以上这种写法在Xilinx是允许的，但是在有些编译器，例如Altera的编译器中是会报错的。

Vivado使用技巧——代码风格

- ⦿ 不建议将置位和复位置于同一个寄存器中。
- ⦿ 避免使用组合环路，当不需要优先编码器时，应使用CASE来降低功耗。

Vivado使用技巧——资源使用

- PLL和MMCM相比，官方建议优先使用PLL，因为其具有更严格的抖动控制。只有当：PLL用尽时；或所需功能MMCM可以提供而PLL不能时才使用MMCM。
- RAM不要使用异步复位，而且建议采用READ_FIRST模式，可以确保同一时钟同时连接到Block RAM的两个端口不会发生存储器冲突。RAM深度小于64的，建议使用LUTRAM，可以实现更加理想的性能；深度大于256的，建议使用Block RAM，将会在最大功率、面积、功耗这三方面更优。Block RAM建议使用NO CHANGE模式以取得最佳的功耗。

Vivado使用技巧——资源使用

- 准备映射到DSP48中的代码应完全流水线化，这样可以利用所有的流水线级。为了灵活使用DSP，应避免使用置位条件，但可以使用复位，只支持同步复位。异步复位会导致欠佳的面积、性能和功耗。
- 全局时钟资源有BUFG\BUFGCE\BUFGMUX\BUFGCTRL四种，BUFGCE用于门控，BUFGMUX用于多时钟切换，BUFGCTRL用于异步控制。

Vivado使用技巧——时序约束

◎ 时序例外的优先级规则

- 1、约束越具体，优先级越高。例如：
 - `set_max_delay -from [get_clocks clkA] -to [get_pins inst0/D] 12`
 - `set_max_delay -from [get_clocks clkA] -to [get_clocks clkB] 10`
- 第一个`set_max_delay`约束具有更高的优先级，因为`-to`选项使用的pin比时钟更具体。
- 2、时序例外优先级如下：
 - 1. `set_false_path`
 - 2. `set_max_delay` or `set_min_delay`
 - 3. `set_multicycle_path`
- 尽管`set_clock_groups`命令相当于两个时钟之间的两个`set_false_path`命令，但不将其视为时序例外。它的优先级高于时序例外。

Vivado使用技巧——工作流程

- ◎ Keep、DONT_TOUCH是非常有价值的设计调试属性，但是在驱动三态输出或者双向信号的层级上不要使用。
- ◎ 反复运行 “*phys_opt_design*” 可以改进时序。
- ◎ 功耗优化：
 - 让功耗分析更加精确的做法有：
 - 时钟约束更加准确；
 - I/O约束更加准确；
 - 顶层控制信号的信号速率和高电平占空百分比更加准确；
 - 在布局之后进行功耗优化。

Vivado使用技巧——器件配置

◎ Vivado主要配置管脚简要说明：

- INIT_B：从低到高的脉冲说明加点初始化完成。配置期间该信号下降说明FPGA器件遇到比特流CRC错误；
- M[2:0]：用于配置模式的选择。配置期间管脚电平值固定不变；
- PROGRAM_B：用于清空FPGA配置存储器并再次开始配置；
- CFGBVS：用于提供电源支持范围1.8~3.3V；
- PUDC_B：用于标识加电后和配置期间是否在SelectIO引脚上启用内部上拉电阻。

◎ Xilinx的比特流文件常用的一个同步字为：AA995566。

目录

- ◎ 1. Vivado软件概述
- ◎ 2. Vivado软件使用基本流程
- ◎ 3. Vivado约束设计基本流程
- ◎ 4. Vivado开发的注意事项
- ◎ *附录1：SDC介绍*
- ◎ 附录2：ISE时序约束设计

附录1: SDC介绍

- ◎ SDC(Synopsys Design Constrain)格式是由Synopsys公司推出的一套时序约束方面的标准格式,主要被IC设计行业广泛采用,被视为默认的行业标准。由于其在IC行业内的统治地位,得到了多种EDA工具的支持,常用做约束文件的扩展名。
- ◎ SDC采用了Tcl工具命令语言,遵循Tcl语法规则。
- ◎ Altera、Actel等FPGA厂家的产品很早就开始采用该格式了。而Xilinx从Vivado系列开始采用该格式作为标准。

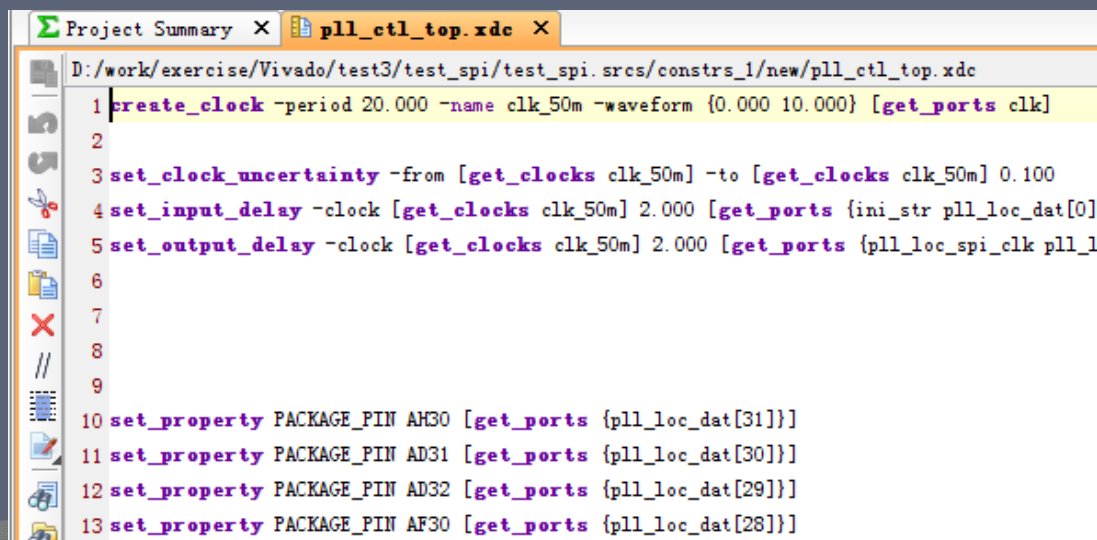
附录1：SDC介绍

- 使用SDC约束，可以在不同的EDA工具软件之间进行平滑衔接。可以将设计的时序约束和物理约束信息很方便地导入新工具。
- 主要组成部分有：
 - SDC版本（可选）；
 - SDC约束单元（可选）；
 - 设计约束；
 - 注释信息（可选）。

附录1: SDC介绍

生成SDC约束文件的方法:

- Synopsys的诸多EDA工具, 如Design Compiler, IC Compiler, Astro, or PrimeTime tools;
- 第三方支持SDC标准的EDA工具;
- 手动编写



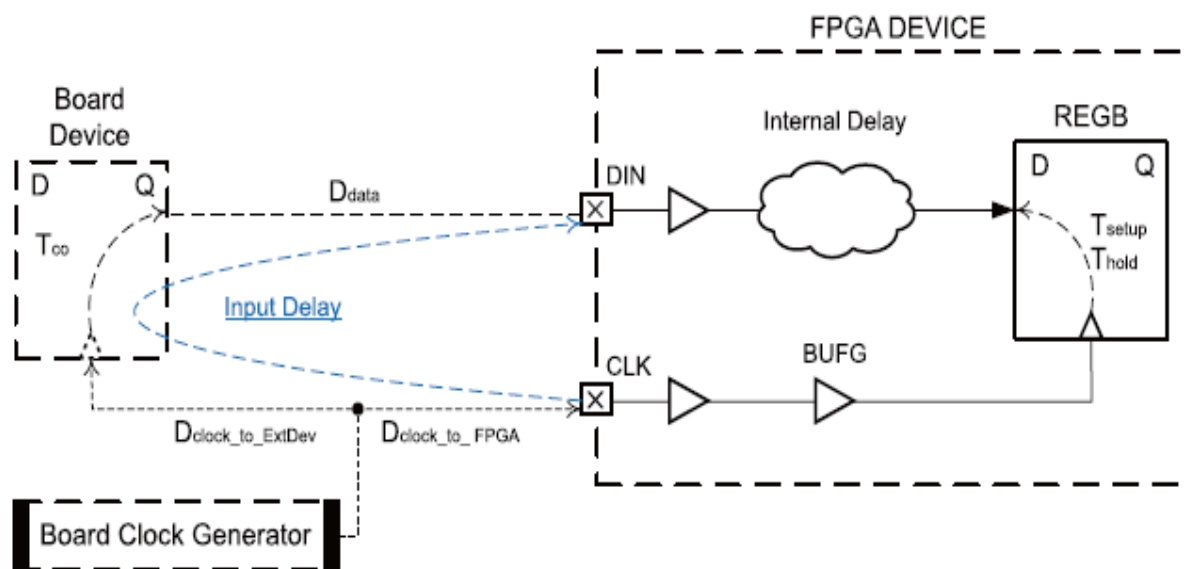
```
Project Summary x pll_ctl_top.xdc x
D:/work/exercise/Vivado/test3/test_spi/test_spi.srcs/constrs_1/new/pll_ctl_top.xdc
1 create_clock -period 20.000 -name clk_50m -waveform {0.000 10.000} [get_ports clk]
2
3 set_clock_uncertainty -from [get_clocks clk_50m] -to [get_clocks clk_50m] 0.100
4 set_input_delay -clock [get_clocks clk_50m] 2.000 [get_ports {ini_str pll_loc_dat[0]}]
5 set_output_delay -clock [get_clocks clk_50m] 2.000 [get_ports {pll_loc_spi_clk pll_1
6
7
8
9
10 set_property PACKAGE_PIN AH30 [get_ports {pll_loc_dat[31]}]
11 set_property PACKAGE_PIN AD31 [get_ports {pll_loc_dat[30]}]
12 set_property PACKAGE_PIN AD32 [get_ports {pll_loc_dat[29]}]
13 set_property PACKAGE_PIN AF30 [get_ports {pll_loc_dat[28]}]
```


附录1: SDC介绍

- ◎ *create_clock -name SysClk -period 10 -waveform {0 5} [get_ports sysclk]*
- ◎ Clock Uncertainty = clock skew + clock jitter
 - Clock skew是同一时钟在不同点的差异;
 - Clock jitter是同一时钟在某一点的差异。
- ◎ 对于抖动:
 - 如果不同数量的抖动进入器件, 可在每个主时钟上使用set_input_jitter命令。
 - 如果器件电源有噪声, 想要调整全局抖动, 应使用set_system_jitter。
 - 对于生成时钟, 抖动源自主时钟和时钟修改模块的特性, 用户不需要调整。

附录1: SDC介绍

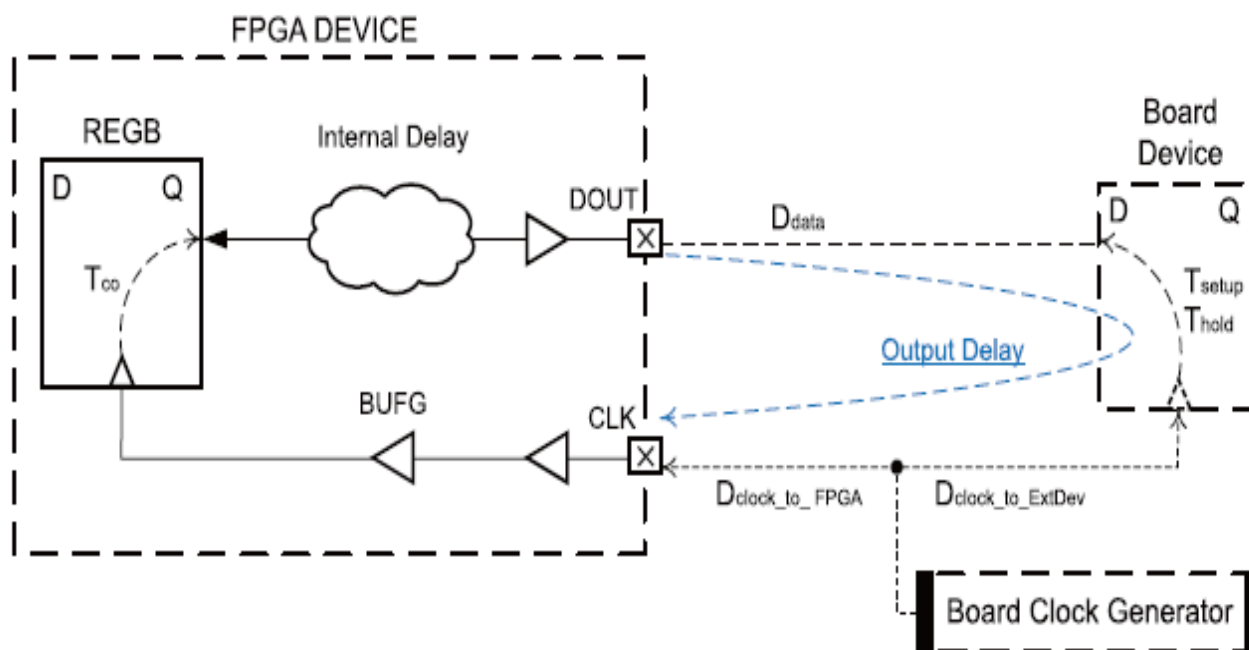
输入延迟定义:



$$\begin{aligned}\text{Input Delay}(\max) &= T_{co}(\max) + D_{data}(\max) + D_{clock_to_ExtDev}(\max) - D_{clock_to_FPGA}(\min) \\ \text{Input Delay}(\min) &= T_{co}(\min) + D_{data}(\min) + D_{clock_to_ExtDev}(\min) - D_{clock_to_FPGA}(\max)\end{aligned}$$

附录1: SDC介绍

输出延迟定义:



$$\text{Output Delay(max)} = T_{\text{setup}} + D_{\text{data}}(\text{max}) + D_{\text{clock_to_FPGA}}(\text{max}) - D_{\text{clock_to_ExtDev}}(\text{min})$$

$$\text{Output Delay(min)} = T_{\text{hold}} + D_{\text{data}}(\text{min}) + D_{\text{clock_to_FPGA}}(\text{min}) - D_{\text{clock_to_ExtDev}}(\text{max})$$

附录1: SDC介绍

Type of information	Commands
Operating conditions	set_operating_conditions
Wire load models	set_wire_load_min_block_size set_wire_load_mode set_wire_load_model set_wire_load_selection_group
System interface	set_drive set_driving_cell set_fanout_load set_input_transition set_load set_port_fanout_number
Design rule constraints	set_max_capacitance set_max_fanout set_max_transition set_min_capacitance

附录1: SDC介绍

Type of information	Commands
Timing constraints	<code>create_clock</code> <code>create_generated_clock</code> <code>group_path</code> <code>set_clock_gating_check</code> <code>set_clock_groups</code> <code>set_clock_latency</code> <code>set_clock_sense</code> <code>set_clock_transition</code> <code>set_clock_uncertainty</code> <code>set_data_check</code> <code>set_disable_timing</code> <code>set_ideal_latency</code> <code>set_ideal_network</code> <code>set_ideal_transition</code> <code>set_input_delay</code>

附录1: SDC介绍

Type of information	Commands
Timing constraints	set_max_time_borrow set_output_delay set_propagated_clock set_resistance set_timing_derate
Timing exceptions	set_false_path set_max_delay set_min_delay set_multicycle_path
Area constraints	set_max_area

附录1: SDC介绍

Type of information	Commands
Multivoltage and power optimization constraints	<code>create_voltage_area</code> <code>set_level_shifter_strategy</code> <code>set_level_shifter_threshold</code> <code>set_max_dynamic_power</code> <code>set_max_leakage_power</code>
Logic assignments	<code>set_case_analysis</code> <code>set_logic_dc</code> <code>set_logic_one</code> <code>set_logic_zero</code>

附录1: SDC介绍

Design object	Access command	Description
design	current_design	A container for cells. A block.
clock	get_clocks all_clocks	A clock in a design. All clocks in a design.
port	get_ports all_inputs all_outputs	An entry point to or exit point from a design. All entry points to a design. All exit points from a design.
cell	get_cells.	An instance of a design or library cell
pin	get_pins	An instance of a design port or library cell pin.
net	get_nets	A connection between cell pins and design ports
library	get_libs	A container for library cells
lib_cell	get_lib_cells	A primitive logic element.
lib_pin	get_lib_pins	An entry point to or exit point from a lib_cell.
register	all_registers	A sequential logic cell.

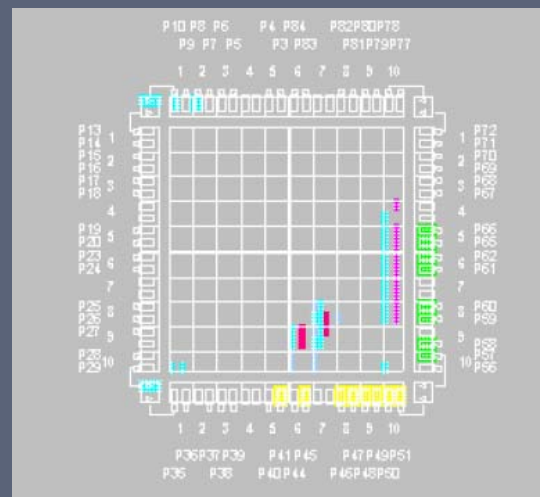
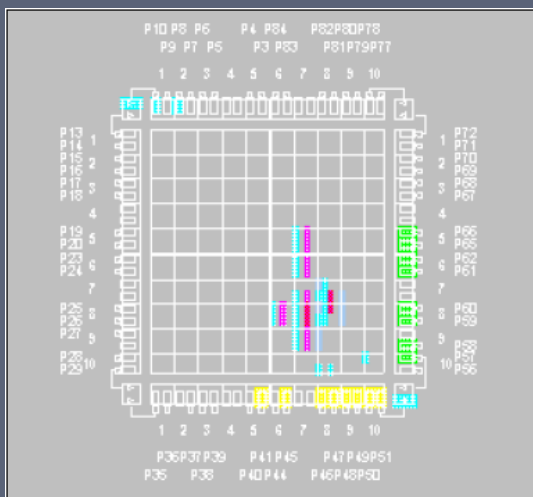
目录

- ◎ 1. Vivado软件概述
- ◎ 2. Vivado软件使用基本流程
- ◎ 3. Vivado约束设计基本流程
- ◎ 4. Vivado开发的注意事项
- ◎ 附录1：SDC介绍
- ◎ *附录2：ISE时序约束设计*

附录2: ISE时序约束设计

● 时序约束对设计有什么影响？

- 实现工具不会试图发现能够获得最佳速度的布局和布线方式。相反，设计实现工具试图满足设定的性能目标。
- 提高性能的途径是将逻辑放得尽可能近，使用尽可能短的布线资源。



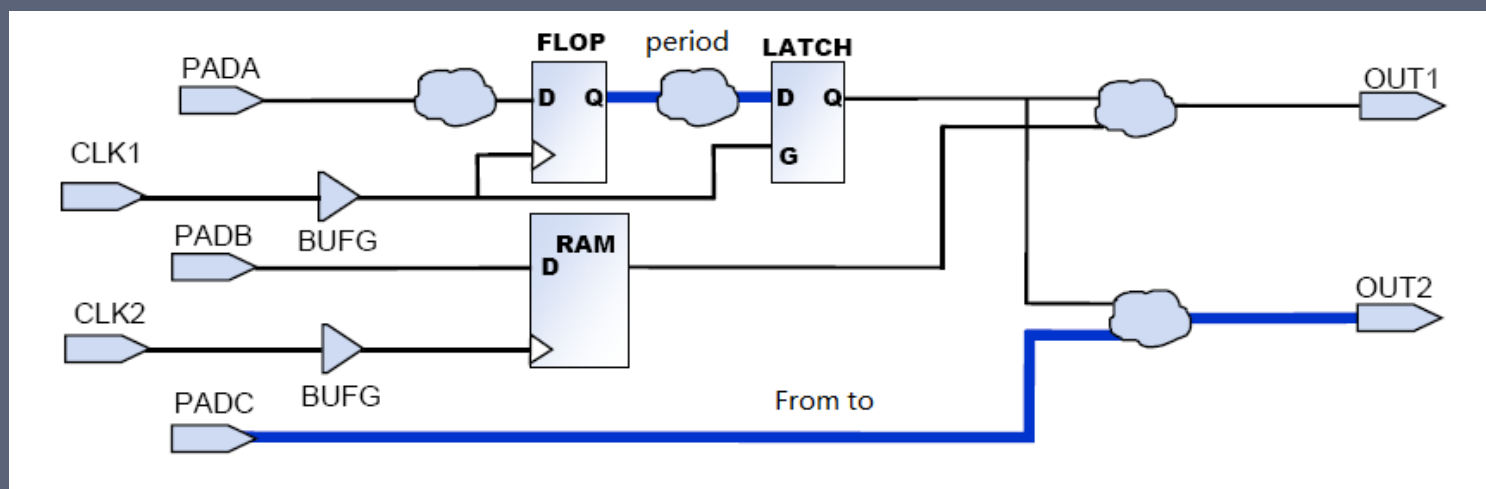
附录2：ISE时序约束设计

- ◎ 时序约束用于定义性能目标
 - 过紧的时序约束会延长编译时间；
 - 不实际的时序约束会导致无法进行实现操作；
 - 可以利用综合报告或静态时序分析报告来判断时序约束是否可行。

附录2: ISE时序约束设计

主要的时序约束类型:

- Period
 - 适用于同一时钟网络中的同步元件之间的路径
- From To (Pad to Pad)
 - 适用于不包含任何同步元件的纯组合延迟路径和异步时钟路径分析

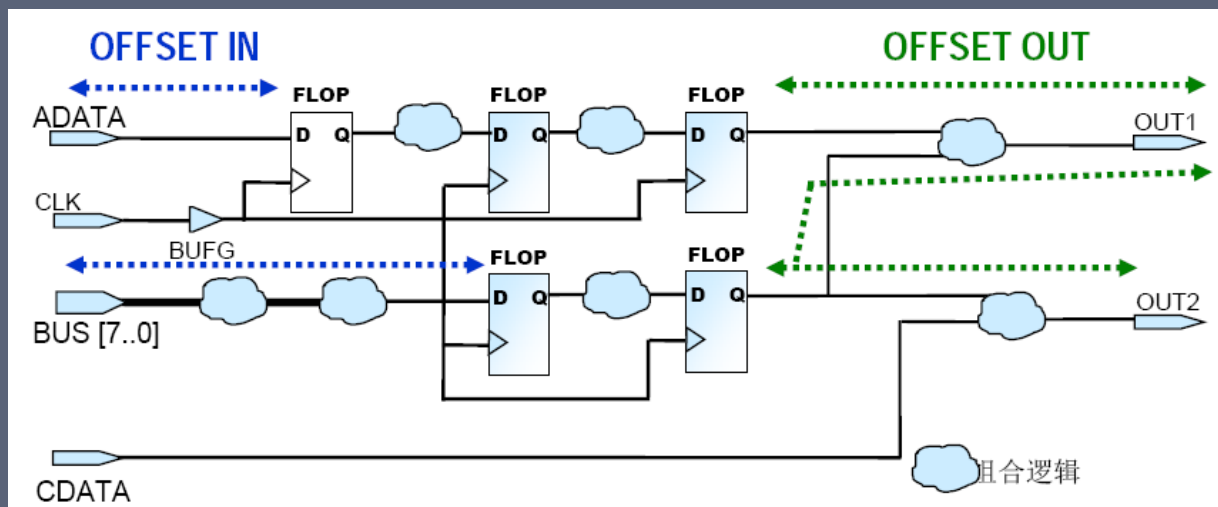


附录2：ISE时序约束设计

主要的时序约束类型：

- Offset IN/OUT

- 从输入引脚到同步元件（OFFSET IN）；
- 从同步元件到输出引脚（OFFSET OUT）。



附录2: ISE时序约束设计

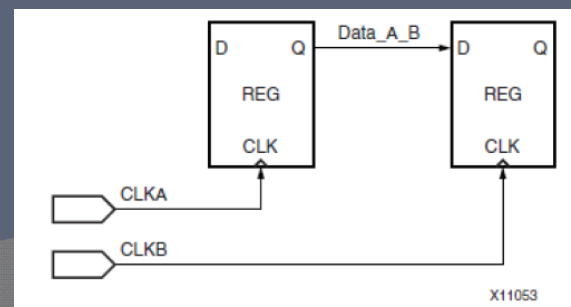
◎ PERIOD 约束语法如下:

- *NET "ClockName" TNM_NET = "TNM_NET_Name";*
- *TIMESPEC "TS_name" = PERIOD "TNM_NET_Name" <PeriodValue> HIGH <HighValue>%;*
- 在PERIOD 约束中, “PeriodValue” 定义了时钟周期的有效时间。“HighValue” 表示时钟波形为 “HIGH” 在一个时钟周期中所占的百分比。例如:
 - *NET "clk" TNM_NET = clk;*
 - *TIMESPEC TS_clk = PERIOD "clk" 20 ns HIGH 50%;*

附录2：ISE时序约束设计

◎ FROM TO时序约束：

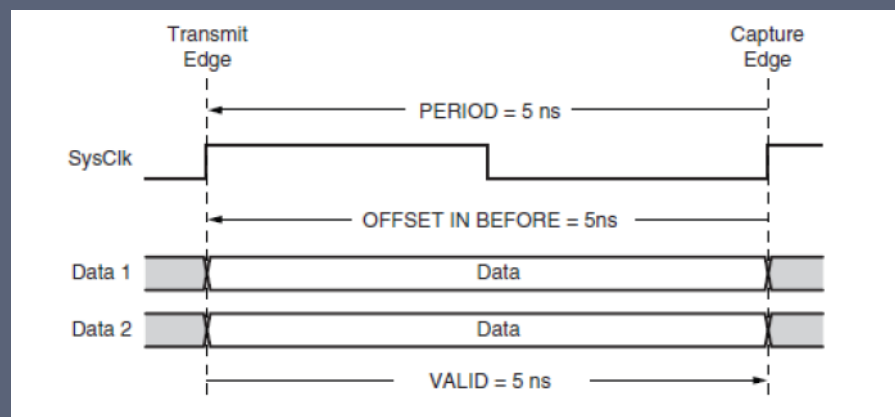
- 具体要求如下：
 - 为源同步元件定义时钟组；
 - 为目的同步元件定义时钟组；
 - 用带“DATAPATHONLY”关键词的FROM-TO在两个时钟组之间定义最大数据路径延时；
- *NET "CLKA" TNM_NET = FFS "GRP_A";*
- *NET "CLKB" TNM_NET = FFS "GRP_B";*
- *TIMESPEC TS_Example = FROM "GRP_A" TO "GRP_B" 5ns DATAPATHONLY;*



附录2：ISE时序约束设计

● OFFSET约束：

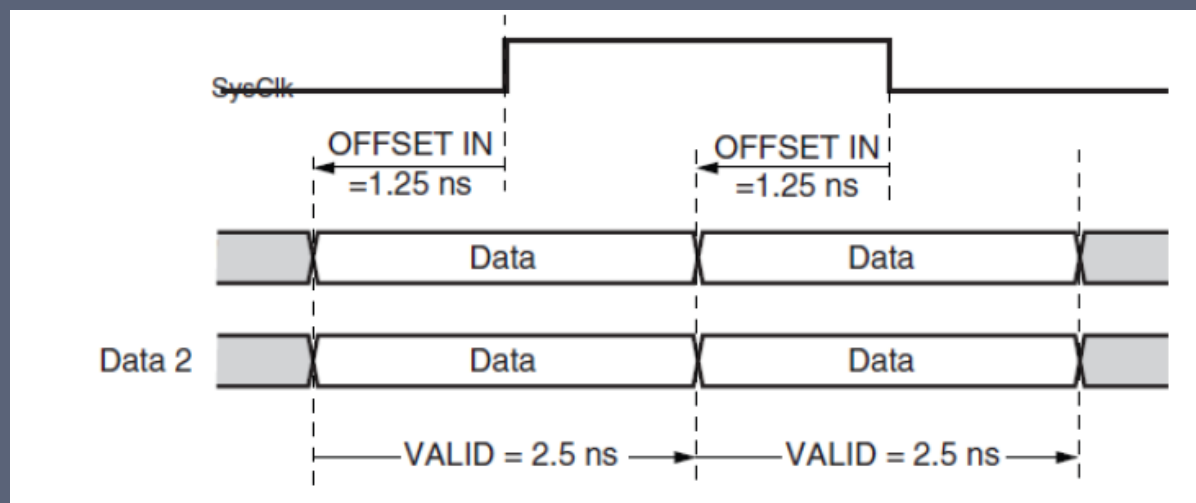
- 添加“OFFSET IN”约束的语法：
- *OFFSET = IN <value> VALID <value> BEFORE clock;*
- 在“OFFSET IN”约束中，IN <value>决定了数据一开始有效的起始时间到抓取数据的时钟沿之间的时间。VALID <value> 决定了数据有效时间。例如：
 - *OFFSET = IN 5ns VALID 5ns BEFORE “SysClk”;*



附录2: ISE时序约束设计

◎ 双数据速率传输 (DDR) 时序约束

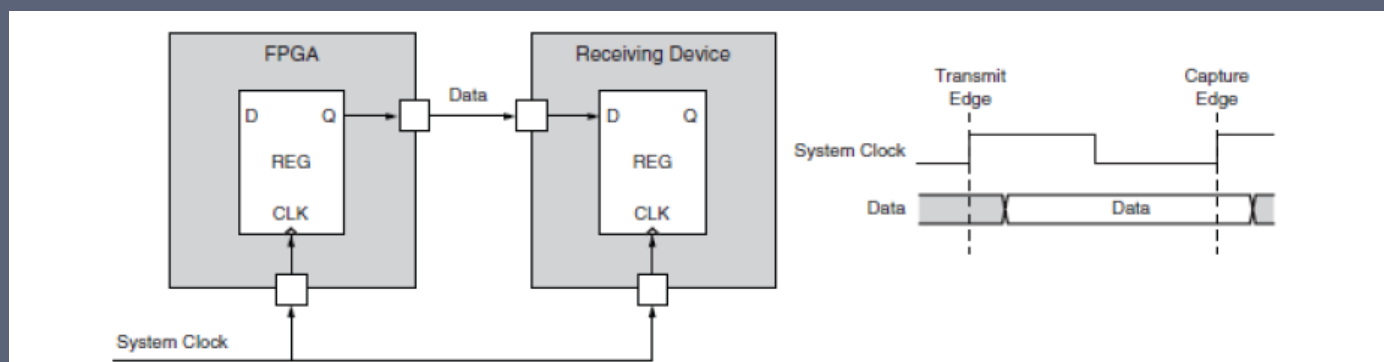
- 对DDR OFFSET IN 的约束如下:
- *OFFSET = IN<value> VALID<value>BEFORE "clock" RISING;*
- *OFFSET = IN<value> VALID<value>BEFORE "clock" FALLING;*



附录2：ISE时序约束设计

● OFFSET约束：

- 添加“OFFSET OUT”约束的语法：
- *OFFSET = OUT <value> AFTER clock;*
- OFFSET OUT 约束中，OFFSET = OUT <value>决定了从FPGA 输入端口时钟上升沿到FPGA输出端口数据开始有效的最大时间。例如：
 - *OFFSET = OUT 5ns AFTER “ClkIn”;*



附录2: ISE时序约束设计

◎ 双数据速率传输 (DDR) 时序约束

- 设置输出时序需要:

- 为输出时钟定义个时钟名 (TNM) 来创建一个时钟组, 这个组包含了所有由输出时钟触发的寄存器;
- 为接口时钟上升沿定义全局OFFSET OUT 约束;
- 为接口时钟下降沿定义全局OFFSET OUT 约束;

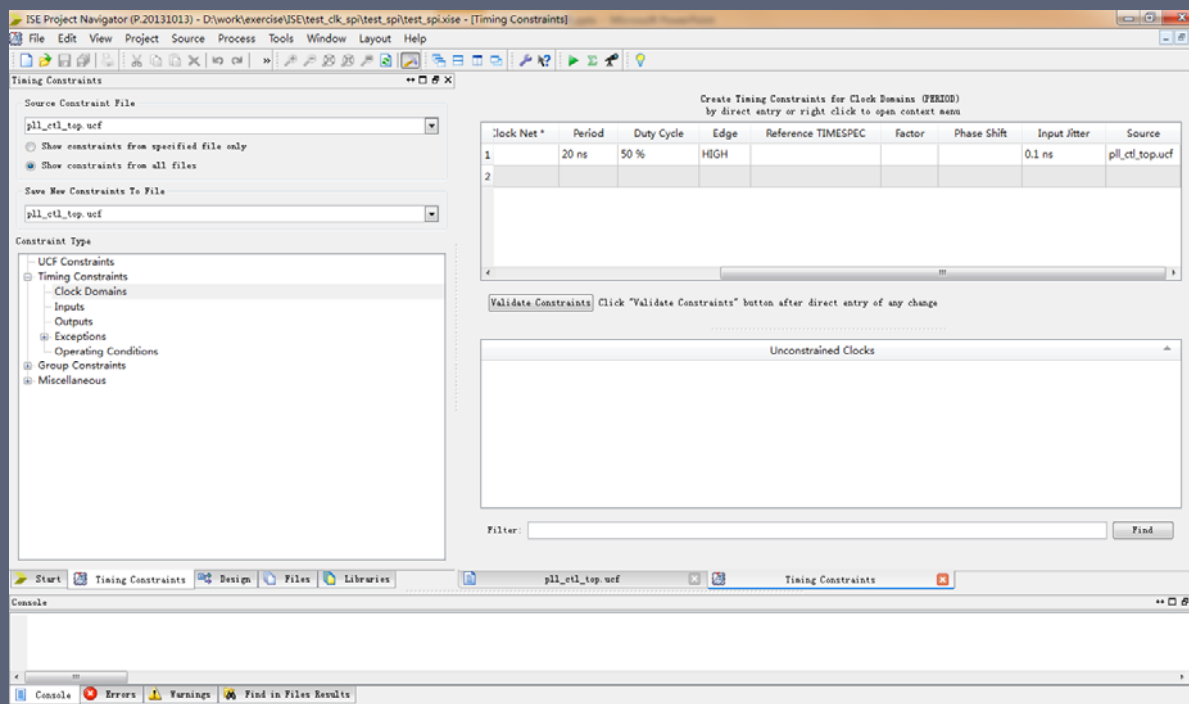
- ◎ 在OFFSET OUT 约束中, OFFSET = OUT<value>决定了从输入时钟端口的上升沿到FPGA输出端口数据开始有效之间的最大时间。关键词REFERENCE_PIN 定义重新生成的输出时钟作为参考点, 而输出数据管脚倾斜的报告就是根据这个参考点生成的。

- *NET "ClkIn" TNM_NET = "ClkIn";*
- *OFFSET = OUT AFTER "ClkIn" REFERENCE_PIN "ClkOut" RISING;*
- *OFFSET = OUT AFTER "ClkIn" REFERENCE_PIN "ClkOut" FALLING;*

附录2：ISE时序约束设计

● 时序约束添加方法：

- 手动建立约束文件，添加约束条件；
- 选择“Tools → Constrains Editor”。



参考文献

- ◎ Ultra FAST设计方法指南；
- ◎ Synopsys约束sdc应用手册；
- ◎ Xilinx_Timing_Constraints_User_Guide。

谢谢！

Q & A